DPaaS: Improving Decentralization by Removing Relays in Ethereum PBS

Chenyang Liu
Duke University
chenyang.liu@duke.edu

Ittai Abraham
a16z crypto
ittaia@gmail.com

Matthew Lentz
Duke University
mlentz@cs.duke.edu

Kartik Nayak
Duke University
kartik@cs.duke.edu

Abstract*—Proposer-Builder Separation (PBS) in Ethereum improves decentralization and scalability by offloading block construction to specialized builders. In practice, MEV-Boost implements PBS via a side-car protocol with trusted relays, resulting in increased centralization as well as security and performance concerns. We propose Decentralized Proposer-as-a-Service (DPaaS), a deployable architecture that eliminates relays while preserving compatibility with Ethereum's consensus layer. Our insight is that we can reduce centralized trust by distributing the combined roles of the proposer and relay to a set of Proposer Entities (PEs), each running in independent Trusted Execution Environments (TEEs). For compatibility, DPaaS presents itself to Ethereum as a single validator, leveraging the threshold and aggregation properties of the BLS signature scheme used in Ethereum. To decentralize the relay roles (e.g., auctioneer) across TEEs, we developed a new Byzantine broadcast protocol that provides necessary censorship-resistance and availability-backed properties on top of standard consensus. We implemented a prototype of DPaaS and validated it end-to-end on a local Ethereum testnet. Our evaluation, deployed across four independent cloud hosts and driven by real-world traces, shows that DPaaS achieves $\leq$ 5 ms bid processing latency, 55.69 ms latency from the end of auction to block proposal, and 3% more MEV earnings – demonstrating that DPaaS can offer security and decentralization benefits while providing strong performance.

I. Introduction

Maximal Extractable Value (MEV) [1], [2] refers to the maximum value that can be extracted from block production by changing the order of transactions in a block. Despite adding economic incentives to DeFi systems like Ethereum, MEV contributes greatly to centralization because it disproportionately favors participants with resources (e.g., trading companies) [3], [4], [5], [6]. This has led to the *Proposer-Builder Separation (PBS)* [5] architecture proposed by Ethereum community: every slot (12 seconds), a randomly selected *validator* acts as the *proposer* to select a block for the slot by soliciting bids from *builders* who try to construct high-value blocks. PBS's vision is to make the market for builders more open and competitive, and ensure that all validators have equal opportunity to capture MEV.

Unfortunately, it is hard to realize PBS in practice without introducing new forms of centralization. First, the implementation must ensure fair exchange [7]: proposers should receive available and valid blocks from builders, which include fee payment to the proposer, while builders should have the confidentiality of their block contents protected unless selected for the slot. Supporting such a fair exchange requires the introduction of a trusted party, increasing centralization. Second, the implementation must deliver strong performance, given that there is a trend towards shorter slot times [8] and that MEV is very sensitive to latency [9], [10]. Any decentralized solution inherently adds latency overhead due to communication. Third, the implementation should be readily (and incrementally) deployable. While it is possible to modify Ethereum's core protocol (e.g., validator block endorsement), any changes typically take years to finalize [11], [12], [13]. Additionally, it is important to maintain compatibility with the broader ecosystem (e.g., builder bid cancellation [14]).

Today, PBS is implemented via MEV-Boost [15], a sidecar protocol that introduces *relays* to serve as trusted auctioneers and escrows. The primary drawback is the inherent trust placed in relays by builders and proposers. There is no underlying mechanism preventing relays from exposing non-winning block payloads to steal MEV [16] or biasing the auction by censoring builders' bids. Additionally, relays are highly centralized, with over 90% of blocks in Ethereum routed through just five relays and 84% controlled by only three companies [17]. Moreover, relays incur performance overheads due to their *commit-and-reveal* scheme, which involves multiple rounds between the proposer and relay to complete the fair exchange. This leads to the proposer capturing less MEV due to ending the auction earlier [10], and could be amplified by shorter slots in the future [8].

Given the growing consensus that centralized, trusted relays should be eliminated, there have been several proposals for PBS [18], [19], [20], [21]. TEE-Boost [20] uses Trusted Execution Environments (TEEs) for builders to prove the validity of their block to a proposer without revealing its contents, but it does not address the block availability. Other proposals [18], [19] modify Ethereum's consensus layer, which makes deployment challenging. A proposal by Paradigm [19] extends TEE-Boost and addresses block availability through the use of Silent Threshold Encryption [22] for the attester committee to decrypt the payload. However, it does not address potential bias in the auction and poses issues with supporting Ethereum's dynamic availability goals. Enshrined PBS (ePBS) [18] proposes staking builders to address fair exchange. Although ePBS is in development, Ethereum's long-term roadmap envisions its adoption. In ePBS, proposers can still source blocks from sidecar protocols, which offer lower entry barriers for builders and often higher profitability [23].

Our insight is that we can remove centralized trust and improve performance by decentralizing the combined roles of the proposer and relay. Based on this insight, we introduce Decentralized Proposer-as-a-Service (DPaaS), a deployable architecture that leverages a distributed set of TEEs. Users enroll as validators through a given deployment that meets their

security policy (e.g., different cloud providers and/or TEEs). DPaaS handles all of the responsibilities, including sourcing blocks from builders, running the auction, and proposing the block. To avoid centralized trust, DPaaS ensures that individual (or a small fraction of) compromised TEEs do not lead to the same loss of security as with relays. By combining the proposer and relay roles, DPaaS reduces the latency between the end-of-auction and proposal and allows the user to capture more MEV from later (higher) bids.

We need to address several challenges to realize DPaaS. First, to preserve full compatibility with Ethereum's existing consensus protocol, DPaaS must appear as a single entity with respect to the rest of the Ethereum network, despite being distributed across multiple TEEs. Second, DPaaS must ensure all security guarantees and provide existing relay functionality (e.g., bid cancellation), despite the presence of a small fraction of faulty TEEs [24] and/or partially synchronous networks [25]. Third, DPaaS should achieve performance comparable to centralized, relay-based systems to ensure practical deployment, despite acting as a decentralized auctioneer.

To summarize, our contributions include:

- We propose Decentralized Proposer-as-a-Service (DPaaS), a deployable implementation of PBS for Ethereum that eliminates centralized relays and is backward compatible.
- We formalize the Censorship-resistant and availability-backed Byzantine Broadcast (Crab-BB) problem, develop a protocol primitive, and build our DPaaS protocol on top to achieve all of the security goals.
- We implement a prototype of DPaaS, which we validated end-to-end in a local Ethereum testnet. We evaluated it using a deployment across four TEEs with diverse cloud providers, availability zones, and TEE implementations. Our evaluation, driven by real-world bid traces, demonstrates DPaaS (on average) processes bids in less than 5 ms and can propose blocks within 55.69 ms once the auction completes (leading to 3% more MEV).

II. Background & Motivation

A. Ethereum and MEV-Boost

Ethereum currently operates under a Proof-of-Stake (PoS) consensus protocol known as Gasper [26]. Participants become *validators* by staking 32 ETH [27]. Ethereum divides time into 12-second *slots*; in each slot, a designated validator (i.e., the *proposer*) selects a block while a committee of validators (*attesters*) vote on the block.

Today, MEV-Boost implements PBS via a sidecar protocol that allows the proposer to outsource block construction and MEV extraction to a market of builders. MEV-Boost uses trusted third parties known as *relays* to support fair exchange between builders and proposers. Relays act as block aggregators, auctioneers, and escrows through several key APIs: `submitBlock` for builders, `getHeader`, and `getPayload` for proposers. Fig. 1 shows an example timeline of a slot, focusing on the winning bid (star). Builders start constructing blocks for slot S during slot $S-1$ and submit their

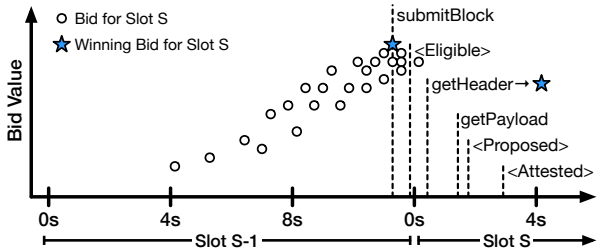

Fig. 1: Example slot timeline with MEV-Boost.

blocks and bids via the `submitBlock` API. After receiving a bid, the relay validates it by simulating the block payload transactions; if successful, the relay includes the bid in the auction (*Eligible*). Eventually, the proposer calls `getHeader` to obtain the block header for the winner of the auction from the relay. The proposer signs the block header and submits it with the call to `getPayload` to obtain the block payload. The relay verifies the signature from the proposer and propagates the Signed Beacon Block (signed block header and beacon block body) [28]; after a delay, typically 1 second, the relay also releases the payload to the proposer so it can propagate the Signed Beacon Block as well. Finally, attesters vote on the block by the 4-second deadline [29].

To eliminate the need for trusted relays, we need an alternative that provides fair-exchange guarantees with comparable performance. While Multi-Party Computation (MPC) and Zero-Knowledge Proofs (ZKPs) can help to reduce trust, both suffer from prohibitive performance overhead. Since bid values increase over slot time [30], [10] and not meeting the strict attestation deadline leads to missed slots [10], minimizing the latency is critical.

B. Trusted Execution Environments (TEEs)

TEEs enable Secure Remote Execution (SRE) on a platform provided by an untrusted infrastructure provider via trusted hardware. TEE-based enclaves or confidential virtual machines (CVMs) can natively run on the CPU with hardware-assisted isolation, which provides confidentiality and integrity protections for application code and data. Additionally, the application owner can determine the identity of the application running via remote attestation. Major cloud providers have adopted many TEE implementations, including Intel SGX [31], Intel TDX [32], and AMD SEV [33].

There are several important considerations regarding the security guarantees. First, there are growing numbers of TEEs' vulnerabilities [34], [35], [36], [37]. Second, TEEs do not address availability concerns, in part due to offloading some roles to untrusted privileged software (e.g., hypervisor).

Using a single TEE serving as a relay still leads to a single point of failure, given vulnerabilities present in the TEE ecosystem. **Therefore, we need to consider a security-in-depth approach, decentralizing trust across a group of (heterogeneous) TEEs**.

C. Threshold Cryptography Schemes

Threshold cryptography primitives are useful to help tolerate a small fraction of TEE failures.

Secret Sharing (SS) A (t, n)-secret sharing scheme [38], [39] enables a dealer to divide a secret among n players so that only groups of at least t players can reconstruct it. In Shamir's Secret Sharing [38], the dealer encodes the secret as a random polynomial and gives each player one share. Any subset of at least t honest players can later combine their shares to recover the original secret via Lagrange interpolation [40].

Threshold Signature Scheme (TSS) A (t, n)-threshold signature scheme (TSS) supports n signers where subsets of size $\geq t$ can produce a signature on a message m, which can be verified using a single public key regardless of the subset. Boneh–Lynn–Shacham (BLS) scheme [41] is a TSS that is heavily used in Ethereum [42]. To support threshold signatures, a (t, n)-secret sharing scheme is used to distribute shares to all n signers. Each signer computes a partial signature on the message, which the aggregator verifies and then interpolates to reconstruct the final signature.

Despite their benefits, threshold cryptographic schemes fundamentally depend on a sufficient number of uncompromised parties' help to recover the secret. **Therefore, we need to pair such threshold schemes with a Byzantine agreement layer**.

D. Byzantine Broadcast

Byzantine broadcast (BB) [43] ensures agreement on a leader's value, matching it if the leader is honest, but applying it to DPaaS is non-trivial. First, the protocol should minimize good-case latency (i.e., synchronous network, honest leader); e.g., $(5f-1)$-psync-VBB achieves optimal two-round latency for $n=4$, $f=1$ [44], [45]. Second, agreement and termination alone are insufficient: **DPaaS requires BB to be censorship-resistant and availability-backed to prevent bid censorship and enable post-agreement reconstruction.**

III. Overview

In this section, we present the design goals of DPaaS (Sec. III-A), provide an overview of the DPaaS system architecture (Sec. III-B), discuss the threat model (Sec. III-C), and design roadmap (Sec. III-D).

A. Goals

The design goals capture the functionality of current relays in the MEV-Boost architecture. However, today all participants must trust relays to enforce these properties correctly, and a compromised or malicious relay can violate one or all of them. In contrast, DPaaS is designed to provide the same relay-equivalent guarantees without relying on unconditional trust in a single party. We summarize the design goals below, focusing on a given slot in which DPaaS instance acts as the proposer.

G1. **Bid Selection** The winning block must have the highest bid among all eligible[1], non-canceled bids.

[1]Eligible bids are a subset of all submitted bids that have been validated and will be considered in the auction. This follows from the definition in relays [46]. We will refine this later in the context of DPaaS in Sec. VI.

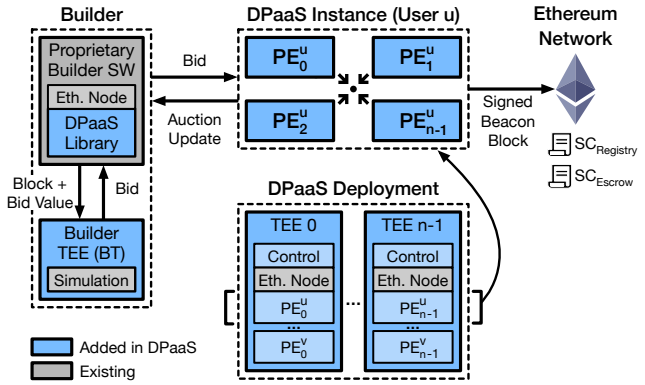

Fig. 2: Overview of DPaaS architecture, showing the interactions for the core online workflow.

G2. **Block Validity** The winning block payload must include the builder's bid as payment to the proposer. Additionally, all honest validators must consider the block valid.

G3. **Block Availability** DPaaS must propose a winning block.

G4. **Failed Trade Privacy** DPaaS must not reveal any non-winning blocks' payload with plaintext transactions.

G5. **Compatibility** DPaaS must not modify Ethereum consensus protocol (e.g., validator duties) and preserve existing builder market's flexibility (e.g., bid cancellation).

G6. **Performance** DPaaS should provide reasonable performance as compared to relays, both from the perspective of builders (e.g., time for a bid to become eligible) and proposers (e.g., time to finalize the auction).

B. Architecture

Fig. 2 shows the architecture of DPaaS. DPaaS realizes the Proposer-Builder Separation (PBS) abstraction in Ethereum in a decentralized manner by executing the combined roles of the proposer and relay across a set of distributed TEEs. A DPaaS deployment consists of a fixed set of n TEEs spread across one or more infrastructure providers; we call any principal setting up and running a DPaaS deployment an *operator*. Each DPaaS deployment hosts an arbitrary number of DPaaS instances, each of which is made up of n Proposer Entities (PEs) across the TEEs. Each TEE contains management software (Control) that enables the operator to spawn a new DPaaS instance when a user wants to enroll as a validator (proposer) in Ethereum through DPaaS. In addition, each TEE operates an Ethereum node to interface with the network for playing validator's role. The PEs in a given instance (e.g., PE_i^u for user u) jointly execute the role of a validator on behalf of the user; our focus, in this paper, is when the validator becomes a proposer for a given slot. Therefore, PEs are responsible for running an auction over bids from builders, determining the winning block, and proposing a Signed Beacon Block to the attester committee by the slot deadline.

On the builder side, most of the software remains unchanged (e.g., receiving transactions, proprietary algorithm for ordering transactions); however, the software must be modified to use the DPaaS Library for handling block submission and auction

updates. After constructing a block, the builder software uses the DPaaS Library to submit the block and a desired bid value to a DPaaS Builder TEE (BT). The BT first performs block validation (e.g., contains payment to proposer, no invalid transactions) by leveraging an existing block simulation library. If successful, the BT creates a bid request containing a sealed payload to the PEs, which serves to: 1) protect the confidentiality of the block payload while allowing later reconstruction via secret sharing, and 2) provide a proof to the PEs that the bid is valid via TEE attestation. The DPaaS Library handles auction updates from individual PEs (e.g., "new highest bid seen by a PE") by aggregating and coalescing them into DPaaS instance-level updates (e.g., "new highest bid that all PEs can agree on").

DPaaS introduces two new smart contracts: registry (SC_R) and escrow (SC_E). SC_R stores metadata for all registered PEs, allowing builders to identify slots with DPaaS proposers and determine the PEs (e.g., endpoint information). SC_E manages user-to-operator payments when blocks are successfully proposed through the operator's deployment, thus providing monetary incentive for operators to maintain availability.

C. Threat Model

There are three main classes of mutually distrusting principals in DPaaS: builders (construct and submit blocks to DPaaS), operators (set up and manage DPaaS deployments), and users (enroll as validators/proposers through DPaaS). The adversary wants to violate any of the security goals (**G1**-**G5**). We assume partially synchronous communication [25], indicating that there exists an unknown Global Synchronization Time (GST), after which communication delays within the network latency are bounded by a known Δ. We assume loosely synchronized clocks (i.e., on the order of milliseconds), which is typical for cloud hosts [47].

Builder. Each builder hosts its own proprietary software (e.g., block construction algorithm) along with the DPaaS Builder TEE (BT) on a cloud provider it trusts. From the perspective of users, any software outside of the BT is completely untrusted and may be compromised by the adversary, while the TEE platform and software inside of the BT are trusted for integrity. For instance, the adversary may attempt to equivocate when sending bids to PEs in the user's DPaaS instance, but they are not able to forge platform attestations or induce unintended changes to the BT code and data at runtime. We consider DoS attacks via Sybil builder bid submission as out of scope; relays are vulnerable to similar attacks, and we could adopt similar reputation mechanisms to address them.

Operator. Since operators manage TEEs from cloud providers, they have the ability to unilaterally terminate them (and thus break availability). We argue that monetary incentives via success-contingent payments (i.e., on block proposal) from enrolled users prevent the operating from doing so.

PEs. The adversary may compromise up to f of the $n \geq 5f - 1$ PEs' confidentiality, integrity, and availability. This assumption is justified by deploying PEs across multiple cloud providers (and/or availability zones), and by using heterogeneous TEE implementations (*e.g.*, Intel SGX/TDX, AMD SEV-SNP, and Arm CCA). We assume major cloud providers will not collude, while availability zones within a provider support fault isolation with independent power and networking infrastructure (reducing the risk of correlated outages). Heterogeneous TEEs further reduce correlated compromise because they rely on different processor vendors, firmware stacks, and isolation mechanisms. Prior attacks [35], [34], [37], [36], [48] showed that many TEE attacks exploit implementation-specific weaknesses in hardware or software. We consider physical attacks [49], [50], [51], [52] against TEEs in scope as one possible approach for PE compromise; however, such attacks are difficult to launch, especially across cloud providers.

This threat model represents a significant improvement over those assumed by relays: a single-domain compromise or arbitrary off-chain operator behavior can silently violate relay security guarantees, whereas DPaaS remains secure as long as at most f PEs are faulty. In DPaaS, operator misbehavior is largely confined to availability degradation and is economically disincentivized, since operators are rewarded only for successful block proposals. We discuss how enrolling users (Sec. IV) and builders (Sec. VI-B) can determine whether a given operator's configuration meets their trust assumptions.

Note on Asymmetry We assume the adversary may violate the integrity of some PEs, up to a bound (f out of n), but not BTs. We want to capture the strongest threat model for which our DPaaS design remains correct. Compromising BTs can only result in missed slots, which are easily detected and mitigated (e.g., blocklist); however, proposer compromise is far more serious (e.g., no privacy or fair exchange guarantees). Recent PBS proposals [19], [20], [53] also introduce TEEs at builders and highlight the fact that bugs in relays are more common than TEE integrity vulnerabilities (in addition to the scope of BT compromise noted above).

D. Design Roadmap

The DPaaS design consists of two phases. In the *offline phase*, the PEs establish an attested validator identity that allows users and builders to verify the deployment (Sec. IV). In the *runtime phase*, the PEs jointly play the combined role of relay and proposer (e.g., running the auction, signing the winning block). A key challenge involves finalizing the auction winner while protecting against censorship of bids and ensuring the availability of shares for winning block reconstruction, which requires more than a standard agreement. We first formalize this sub-problem, censorship-resistant and availability-backed Byzantine Broadcast (Crab-BB), and provide a generic protocol primitive, $\Pi_{\text{Crab-BB}}$, that solves it (Sec. V). Afterwards, we describe the DPaaS runtime, including the auction protocol and how we build on top of this primitive with PBS-specific semantics (Sec. VI).

IV. DPaaS Design - Offline Setup

In this section, we will first describe the overview of the offline workflow for setting up a verifiable DPaaS instance,

Algorithm 1 User Enrollment

D_i = Deployment on TEE_i, U = User, O = Operator

1: U $\rightarrow$ O: Sends enrollment request
2: O $\rightarrow$ D_i: Sends instance request with base manifest M^b
   $M^b = \{m_j^b\}$ and m_j^b = [Endpoint$_j$, Placement$_j$]
3: D_i: Starts a new PE_i process with M^b as input
4: PE_i: Generate two key pairs
   $(sk_{\text{V}}^i, pk_{\text{V}}^i)$ for the validator signing key
   $(pk_{\text{asym}}^i, sk_{\text{asym}}^i)$ for secret shares from builders
5: $\mathsf{PE}_i \rightarrow \mathsf{PE}_j$: Connect to all other PE_j via RA-TLS
   Attested: m_j^b, measurement$_i$
6: $\mathsf{PE}_i \rightarrow \mathsf{PE}_j$: Send pk_{V}^i and pk_{asym}^i to all other PEs
7: $\mathsf{PE}_i \leftrightarrow \mathsf{PE}_j$: Participate in DKG to generate key pair
   $(sk_{\text{p\_sign}}^i, pk_{\text{p\_sign}}^i)$ for threshold signatures
8: PE_i: Generate the local manifest m_i and Sign(sk_{V}^i, m_i)
   m_i = [Endpoint$_i$, Placement$_i$, pk_{V}^i, pk_{asym}^i]
9: $\mathsf{PE}_i \rightarrow \mathsf{PE}_j$: Send m_i and Sign(sk_{V}^i, m_i) to all other PEs
10: PE_i: Construct full manifest $M = \{(m_i, \text{Sign}(sk_{\text{V}}^i, m_i))\}$
11: $\mathsf{PE}_i \rightarrow D_i \rightarrow O$: Return M and σ_i = Sign(sk_{V}^i, M)
12: O $\rightarrow$ U: Return M and $\sigma = \sum_{i=0}^{n-1} \sigma_i$ to the user
13: U $\rightarrow$ PE_i: Connect to all PE_i via RA-TLS
   Attested: m_i, measurement$_i$
14: U: Compute aggregate $pk_{\text{V}} = \sum_{i=0}^{n-1} pk_{\text{V}}^i$ and validate σ
15: U: Generate withdrawal key pair $(sk_{\text{W}}, pk_{\text{W}})$
16: U $\rightarrow$ SC_D: Transaction Deposit(pk_{W}, pk_{V})
17: U $\rightarrow$ SC_R: Transaction Register(pk_{V}, M, sign(sk_{W}, "Reg"))
18: U $\rightarrow$ SC_E: Transaction Escrow(pk_{W}, pk_{V})
19: U $\rightarrow$ O: Sends enrollment finalization message
20: O: Checks state of SC_R, SC_E, and SC_D

user enrollment, and verification from the user/builders, then we will elaborate on the key components among the steps.

A. Overview

Any principal can become a DPaaS operator by provisioning and operating a deployment. The operator chooses the size n and deploys n TEE-backed hosts through cloud providers. To realize the failure-isolation assumptions in our threat model, the operator should place these hosts across independent providers (*e.g.*, AWS, Azure, GCP), availability zones (*e.g.*, US East 1 and US East 2), and TEE implementations (AMD SEV-SNP and Intel TDX). The operator acts only as an infrastructure coordinator: it provisions hosts and loads the attested DPaaS VM image, which includes the control software, Ethereum node, and PE application. Users and builders will reject deployments that do not: 1) execute a known image, 2) match the advertised placement information, and 3) conform to their trust assumptions (e.g., with respect to fault isolation).

We present the enrollment workflow in Algorithm 1. Initially, this starts with the user sending a request to enroll with the deployment operator (Line 1). The operator sends a request to the control software running in each TEE in the deployment to launch a new PE with a base manifest that includes endpoint (i.e., IP address and port) and placement (i.e., provider, region, zone) information (Lines 2-3). The launched PEs start by generating two key pairs: one to serve as the partial key for the aggregate validator key, and another for receiving encrypted bids from builders (Line 4). The PEs will connect to each other through RA-TLS and perform mutual attestation based on the contents of the base manifest M^b and a known measurement (Line 5). If successful, the PEs will exchange the public parts for their generated keys (Line 6) and then execute (n, n)-distributed key generation (DKG) to enable threshold signatures (Line 7). Each PE constructs its local manifest m_i, signs and exchanges it with all other PEs, and constructs the full manifest M (Lines 8-10). Afterward, each PE returns M along with its signature σ_i to the operator, who in turn sends it to the user with the aggregate signature σ (Lines 11-12). At this point, the user connects to all of the PEs and validates that their attestations match the data in the full manifest along with a known measurement (Line 13). After the user is convinced, they compute the public part of the aggregate signing key pk_{V} via the partial keys from each PE, and generate a withdrawal key pair $(sk_{\text{W}}, pk_{\text{W}})$ that they will use for becoming a validator (Lines 14-15). Finally, the user issues three smart contract transactions: 1) deposit to become an Ethereum validator with signing key pk_{V}, 2) registering the manifest file to the pk_{V} so that builders can identify PEs for a given slot proposer, and 3) set funds aside in escrow for the operator to claim after successful block proposals (Lines 16-18). The operator subsequently checks the state of these three smart contracts, ensuring that the user has successfully completed their enrollment correctly (Lines 19-20); for instance, that pk_{V} serves as the validator key for an active validator in SC_D.

B. Details

Remote Attestation (Lines 5 and 13) Through remote attestation, users can verify the DPaaS instance configuration, and all PEs in the same DPaaS instance can mutually attest to one another. For mutual attestation, every PE establishes connections to all of these PEs through RA-TLS and verifies four basic requirements: 1) the PE runs on valid trusted hardware, 2) the PE's image including PE software matches a known, correct measurement, 3) the PE is deployed by the correct cloud provider, and 4) the PE has a specific placement within the cloud provider's infrastructure (*e.g.*, region, availability zone). We set the user data field in the attestation (e.g., report_data in AMD SEV-SNP) to be the hash of the base manifest (m_i^b). The same procedure follows for the user, except the user data field is set to the signed local manifest (m_i and Sign(sk_{V}^i, m_i)), which includes the generated public keys.

Validator Keys (Lines 4, 7, 14, 15, 16) In Ethereum, each validator can possess two distinct BLS key pairs [54]: 1) a validator key pair $(sk_{\text{V}}, pk_{\text{V}})$ for validator operations (e.g., block proposal and attestation), and 2) a withdrawal key pair $(sk_{\text{W}}, pk_{\text{W}})$ for controlling access to funds. DPaaS leverages this separation by making PEs generate and manage validator private key sk_{V} for operational duties, while allowing users to generate and retain sole control over the withdrawal key sk_{W} (and thus their underlying funds).

Each PE_i first generates $(sk_{\text{V}}^i, pk_{\text{V}}^i)$ independently. Moreover, to support threshold signatures [55] (with $t = f + 1$), each party generates a partial BLS key pair $(sk_{\text{p\_sign}}^i, pk_{\text{p\_sign}}^i)$

which enables interpolating the signature from any t PEs to that of a signature over the same message with sk_{V} (without any PEs knowing sk_{V}). DPaaS employs (n, n)-distributed key generation (DKG) protocol [56], [57] to achieve that without relying on any trusted third party. Since this only takes place once during initial setup, we consider n out of n appropriate; if a PE misbehaves, leading to abort, then the procedure repeats until all PEs obtain shares that pass the consistency checks. Finally, each PE_i generates a public-secret key pair $(pk^i_{\text{asym}}, sk^i_{\text{asym}})$ independently, which builders will use to encrypt secret shares during the auction (see Sec. VI-B).

Even though DPaaS is decentralized, the rest of the Ethereum network sees it as a single entity with the $(pk_{\text{V}}, pk_{\text{W}})$ identity. Moreover, sk_{V} is never exposed to any PEs or the user, and the withdrawal key sk_{W} is only known to the user.

Contracts (Lines 16, 17, 18, 20) SC_D refers to Ethereum's official deposit contract. A user enrolls as a validator through DPaaS by depositing with a withdrawal key they control (pk_{W}) tied to the aggregate signing key from DPaaS PEs (pk_{V}). SC_R refers to the DPaaS registry contract, which stores metadata for PEs. It supports a Register API, which allows deposited validators to associate a public signing key pk_{V} with a manifest M; the contract validates the signature against the pk_{W} and verifies if it belongs to a validator. This enables all parties to read the metadata locally with no additional cost (e.g., builders can determine PE endpoints for a DPaaS proposer slot). SC_E refers to the DPaaS escrow contract, deployed by an individual operator, which manages payments from enrolled users. It supports a Escrow API, which allows a user to transfer funds to the contract associated with a given pk_{W} and pk_{V}. It also supports a Claim API, which allows an operator to recover a reward from these funds whenever a block proposed by pk_{V} is committed to the chain. We include additional details on these smart contracts in Appendix G.

V. Censorship-resistant and Availability-backed Byzantine Broadcast (Crab-BB)

In this section, we abstract the crux of decentralized auctioneer as a censorship-resistant and availability-backed Byzantine Broadcast (Crab-BB) problem. We then present our $\Pi_{\text{Crab-BB}}$ protocol that solves this problem, serving as an efficient and secure primitive that we employ in DPaaS.

A. Motivation: Challenges in DPaaS

The key challenge in the runtime phase of DPaaS is determining the winning block, which all honest PEs must agree on to jointly reconstruct, sign, and propose. Builders submit bids continuously to all PEs, which should be considered until the auction deadline; however, due to varying network latency and clocks (or even malicious builders), each PE may observe a different set of bids. More importantly, Byzantine PEs may try to censor certain bids, which motivates the need for the protocol to be *censorship-resistant*: any bid considered eligible by all honest PEs must be considered during winner finalization. Additionally, Byzantine PEs may vote for a winning block but not be willing to participate in reconstruction, which motivates the need for the protocol to be *availability-backed*: there are sufficient honest PEs to reconstruct the block even when f PEs are compromised. Finally, given slot deadlines and MEV opportunities, this process must be as efficient as possible (i.e., minimal rounds of communication).

Given that other applications may desire these properties, we consider the general problem of censorship-resistant, availability-backed Byzantine Broadcast (Crab-BB):

Definition 1. *In the censorship-resistant, availability-backed Byzantine Broadcast (Crab-BB) problem, there are n replicas $P_0, \ldots, P_{n-1}$, of which one is designated as the leader P_ℓ. Furthermore, there are m clients $c_0, \ldots, c_{m-1}$. Each client c_j can have an input set U_j. Each replica P_i collects a set E_i, commits on an evidence value* Val*, and then produces an output set W. The following properties pertain to a given protocol Π with input preparation* Compress *and output interpretation* Interp *for the Crab-BB problem.*

- ***Agreement.*** *No two non-faulty replicas P_i disagree on the output subset W from Π.*
- ***Non-triviality.*** $\text{Val} \neq \perp$*, and must satisfy external validity.*
- ***Termination.*** *All correct replicas eventually decide.*
- ***Timely Termination.*** *After* GST*, all correct replicas decide within a bounded time.*
- ***Censorship-Resistant.*** *If* $\text{GST} = 0$*, then for all clients c_j that delivered their input $u \in U_j$ to all honest replicas (i.e., $u \in E_i$ for all honest replicas i), it holds that $u \dot{\in} W$.*
- ***Availability-backed.*** *For every $u \dot{\in} W$ output from Π, there must exist $f + 1$ honest P_i with $u \in E_i$.*

This general definition uses a new relationship $\dot{\in}$, which we term *implicitly in*. We write $u \dot{\in} V$ to mean that the summary V represents object u. This representation may be explicit (e.g., set membership) or implicit (e.g., application-specific compression rule). Unlike standard Byzantine broadcast, Crab-BB's censorship-resistance and availability-backed guarantees depend on the way replicas process their input set E_i and committed evidence Val. We therefore parameterize Crab-BB by two deterministic functions, Compress and Interp, which respectively prepare replica inputs and interpret the agreed evidence. The next subsection (Sec. V-B) formalizes the requirements these functions must satisfy.

B. Generalized Crab-BB Protocol

We present our $\Pi_{\text{Crab-BB}}$ in Fig. 3. Step 1 is a deterministic function for summarizing what each replica received. Step 2-Step 4 is the consensus layer, which invokes a Validated Byzantine Broadcast (VBB) protocol, denoted by Π_{vbb}, as a subroutine. The external validity of Π_{vbb} is defined by Algorithm 3 in Appendix B. In our setting, we use the $(5f-1)$-psync-VBB protocol [45], achieving two-round good-case latency at the minimum resilience threshold[2]. Step 5 is the output interpretation layer implemented to get the final agreed output set in a deterministic way. The protocol starts

[2]We note that for $n \leq 5f - 2$, using two rounds is impossible and one additional round is necessary [45].

Input: Input set E_i; two functions: Interp, Compress.
Output: Interpreted output W.

The steps are for a given P_i:

S1. Input Preparation. Generate: $V_i \leftarrow \mathsf{Compress}(E_i)$

S2. Exchange-1. Broadcast $\langle \texttt{ex-1}, V_i \rangle_i$.

S3. Exchange-2. At time $T_e + \Delta$, proceed as follows:

- **a.** If have received $> n - f$ distinct validly signed `ex-1` messages, let SQ_i denote the set of all such messages, and broadcast $\langle \texttt{ex-fast-2}, SQ_i \rangle_i$.
- **b.** Otherwise, wait until it has received $n - f$ distinct validly signed `ex-1` messages. Let WQ_i denote the set of all such messages, and broadcast $\langle \texttt{ex-normal-2}, WQ_i \rangle_i$.

S4. Entering $\Pi_{vbb}(\mathcal{F})$. Wait until one of the following conditions holds:

- **a.** received $f + 1$ distinct validly signed `ex-normal-2` messages; or
- **b.** received one validly signed `ex-fast-2` message.

Enter Π_{vbb} with the corresponding evidence as input then executes Π_{vbb} until it commits a value Val.

S5. Output Interpretation. Apply a deterministic interpretation function $W \leftarrow \mathsf{Interp}(\text{Val})$

Fig. 3: $\Pi_{\text{Crab-BB}}$ with configurable input and output interpretation Interp for P_i. $\langle\rangle_i$ means a message signed by P_i.

with the input set E_i to each replica. To begin with, we require Compress to be representation-preserving: for every local input set E_i and every object $u \in E_i$, $u \stackrel{\bullet}{\in} \mathsf{Compress}(E_i)$.

Then each replica broadcasts a signed `ex-1` message containing its local set E_i (Step 2). During the subsequent Δ time, replicas collect `ex-1` messages from their peers. If a replica receives $> n - f$ distinct validly signed `ex-1` messages, it forms a strong quorum certificate, denoted by SQ (Step 3a). Otherwise, it continues waiting until it receives $n - f$ distinct validly signed `ex-1` messages, at which point it forms a weak quorum certificate, denoted by WQ (Step 3b). In either case, once a replica forms a certificate, it signs and broadcasts it.

Next, replicas wait until they obtain either (i) one SQ or (ii) $f + 1$ distinct WQs. An honest replica enters Π_{vbb} after receiving one of these two valid inputs, and uses the corresponding evidence as its input to the broadcast protocol (Step 4). Assuming correctness of Π_{vbb}, all honest replicas eventually commit the same valid output, which must be either a single SQ or a set of $f + 1$ distinct WQ values proposed by some replica. Given a committed evidence value Val, let $\mathcal{C}(\text{Val})$ be the collection of summaries extracted from Val. The threshold-supported object set $\mathsf{Supp}_{2f+1}(\text{Val})$ is equal to:

$$\left\{ u \;\middle|\; |\{V_j \in \mathcal{C}(\text{Val}) : u \stackrel{\bullet}{\in} V_j\}| \geq 2f + 1 \right\}.$$

The interpretation function outputs a canonical summary of this set: $\mathsf{Interp}(\text{Val}) = \mathsf{Compress}(\mathsf{Supp}_{2f+1}(\text{Val}))$.

Correctness We leave the detailed proof of why $\Pi_{\text{Crab-BB}}$ solves the Crab-BB problem to Appendix B, but describe the intuition here. Agreement follows directly from the agreement of Π_{vbb}: all honest replicas commit the same value Val, and $\Pi_{\text{Crab-BB}}$ applies the deterministic interpretation rule (Step 5) to obtain the same output W. Termination follows because, after GST, the two exchange steps of $\Pi_{\text{Crab-BB}}$ complete within at most 2Δ, after which all honest replicas enter Π_{vbb}; the termination of Π_{vbb} then implies that all honest replicas eventually output. Finally, non-triviality follows from the external validity of Π_{vbb}. If the committed value is proposed by an honest leader, it is constructed according to the protocol and satisfies $\mathcal{F}$ defined in Algorithm 3. Otherwise, Π_{vbb} guarantees that any committed value satisfies $\mathcal{F}(\text{Val}) = \textsc{True}$. Since $\mathcal{F}$ accepts only values carrying valid evidence, the output W is non-trivial.

At a high level, the censorship-resistance relies on the two exchange rounds preparing for the input to the Π_{vbb}. Consider a client input u delivered to all honest replicas. First, all honest replicas receiving u ($u \in E_i$) can broadcast V_i and $u \stackrel{\bullet}{\in} V_i$ at the first round. Optimistically, if a SQ is formed, then at least $n - f + 1$ V_i can be included to the committed Val. Among $n-f+1$ V_is, there are $n-f+1-f = 3f$ must be from honest replicas. Because $3f \geq 2f + 1$, every honest replica should interpret the output W to implicitly include u. When only WQ can be formed due to f Byzantine replicas (*i.e.*, keep silent at the exchange rounds) or asynchronous network, binding the $f + 1$ threshold for such WQ to external validity can ensure all at least one honest replica's WQ can be included in which sufficiently many ($\geq n - f = 4f - 1$) honest replicas' V_i can be included. Again, since $4f - 1 \geq 2f + 1$, implicit inclusion of such client input can be guaranteed.

The backed-availability relies on the inclusion threshold of $2f + 1$. Once a client input is implicitly supported by $2f + 1$ replicas, at least $2f + 1 - f = f + 1$ honest replicas must have received (backed) such client input.

VI. DPaaS Runtime Protocol

In this section, we formalize PBS auction semantics and present the runtime protocol Π_{DPaaS}. This achieves the application-level goals (Sec. III-A) by building on the set up (Sec. IV) and using $\Pi_{\text{Crab-BB}}$ (Sec. V).

A. PBS Scheme

We denote the builders participating in the slot as $\mathcal{B}$, where each $\mathtt{b} \in \mathcal{B}$ (identified by public key) submits one or more bids from the start of previous slot ($t = -12$s) to this slot's attestation deadline ($t = 4$s). For each PE_i, we refer to the set of bids it receives from builders as R_i. For simplicity, we define each bid visible to every PE as a tuple $(\mathtt{b}, \mathtt{bv}, \mathtt{seq}, \mathtt{h}, \mathtt{att}, \mathtt{spl})$, which respectively refer to the builder, bid value (in $\mathbb{R}^*$), sequence number (in $\mathbb{N}$), block hash, TEE attestation generated by BT, and sealed payload. For any set of bids S, we refer to the subset from builder $\mathtt{b}$ as $S[\mathtt{b}]$.

We define bid eligibility in DPaaS. In status-quo MEV-Boost, a relay marks a bid as eligible once its block is validated. In contrast, DPaaS runs auctions in a decentralized setting, which necessitates distinguishing between eligibility local to a given PE_i versus globally across PEs. Locally, for each builder, PEs treat the highest-sequence eligible bid as active; thus, bid cancellation [14] is supported by canceling

lower-sequence stale bids. If a builder submits malformed sequence numbers, those bids fail the eligibility checks and are excluded from winner selection. Globally, eligibility and activeness is with respect to $\mathcal{E}_i$ for honest PEs. Bids seen by all honest PEs are guaranteed globally eligible, while others may also qualify with support from Byzantine PEs. However, every bid considered globally eligible must be supported by at least $f+1$ honest PEs, and any bid locally eligible at all honest PEs is globally eligible.

Definition 2. *In the DPaaS instance with protocol Π_{DPaaS}, we can define the following eligibility concerning bids:*

- ***Local Eligibility.*** *Given received bid set R_i, the set of PE_i's locally eligible bids $\mathcal{E}_i$ is:*

$$\mathcal{E}_i = \big\{r \mid r \in R_i \wedge \mathsf{PE}_i \text{ has verified } r.\mathtt{att} \wedge \forall s \in [0, r.seq), \exists r' \in \mathcal{E}_i[r.\mathtt{b}] : r'.seq = s\big\}$$

- ***Local Active.*** *Given locally eligible bid set $\mathcal{E}_i$, the set of PE_i's locally active (non-canceled) bids $\mathcal{V}_i$ is:*

$$\mathcal{V}_i = \bigcup_{\mathtt{b}\in\mathcal{B}} \underset{e\in\mathcal{E}_i[\mathtt{b}]}{\arg\max}(e.\mathtt{seq})$$

- ***Global Eligibility.*** *Given all PE_i's locally eligible bids $\mathcal{E}_i$ and honest PEs' set $\mathcal{HP}$, we define globally eligible bid set $\mathcal{E}$ as $\check{\mathcal{E}} \subseteq \mathcal{E} \subseteq \hat{\mathcal{E}}$ where*

$$\check{\mathcal{E}} = \{e \mid n - f \le |\{i \in \mathcal{HP} \mid e \in \mathcal{E}_i\}|\}$$
$$\hat{\mathcal{E}} = \{e \mid f + 1 \le |\{i \in \mathcal{HP} \mid e \in \mathcal{E}_i\}|\}$$

- ***Global Active.*** *The set of globally active bids is the bid with highest sequence number with respect to every builder.*

$$\mathcal{A} = \bigcup_{\mathtt{b}\in\mathcal{B}} \underset{e\in\mathcal{E}[\mathtt{b}]}{\arg\max}(e.\mathtt{seq})$$

We formalize Goals 1–4 and the bid-cancellation aspect of Goal 5 (see Sec. III-A) through the following composite security definition.

Definition 3. *A protocol Π satisfies DPaaS security goals if, for every slot and every f-bounded adversary, the following hold under partial synchrony with auction deadline T_e and proposal deadline T_p. After* GST*, builder-*PE *latency is bounded by Δ_{b2p}, and bid processing latency in* PE *is bounded by Δ_{pr}.*

3.1 ***Bid Selection Correctness*** *(**G1.**). If $T_e \ge$ GST, then the winning bid w is the globally active bid with the highest bid value: $w = \arg\max_{e\in\mathcal{A}} e.\mathtt{bv}$*

3.2 ***Block Validity*** *(**G2.**). For the winning bid w, the reconstructed payload $w.\mathtt{pl}$ pays at least $w.\mathtt{bv}$ to the enrolled user, and honest Ethereum validators accept the payload as valid.*

3.3 ***Block Availability*** *(**G3.**). If at least one valid bid becomes eligible at all honest* PEs*, then* PE*s jointly reconstruct and produce the corresponding signed beacon block.*

3.4 ***Timely Block Availability*** *(**G3.**). If at least one valid bid becomes eligible at all honest* PE*s before the cutoff T_p, then* PE*s jointly reconstruct and produce the corresponding signed beacon block before T_p.*

3.5 ***Failed Trade Privacy*** *(**G4.**). For each slot, for every non-winning bid, no information beyond payload size and submission timing is released to any party.*

3.6 ***Cancellation Effectiveness*** *(**G5.**). For each builder $\mathtt{b} \in \mathcal{B}$, let e be the most recent bid from $\mathtt{b}$ observed after* GST *and before $T_e - \Delta_{b2p} - \Delta_{pr}$. Then every earlier bid $e' \in \mathcal{E}[\mathtt{b}]$ with $e'.\mathtt{seq} < e.\mathtt{seq}$ is considered canceled and cannot be selected as the winner.*

Input: $\mathtt{slot}, T_e$ **Output:** $\mathtt{SignedBeaconBlock}$

P1. Auction.
- **a.** PE_i starts accepting bids one slot before $\mathtt{slot}$.
- **b.** Until T_e, accept bids from BTs and insert each received input into $\mathcal{E}_i$ if $\textsc{TEEVerify}(r.\mathtt{att}) = \textsc{true}$ and $\textsc{SanityCheck}(r) = \textsc{true}$.

P2. Winner finalization.
- **a.** Agree on $\mathcal{A} \leftarrow \Pi_{\mathsf{Crab\text{-}BB}}(\mathcal{E}_i, \mathsf{AppCompress}, \mathsf{AppInterp})$.
- **b.** Find the winner $w \leftarrow \arg\max_{a\in\mathcal{A}}(a.\mathtt{bv})$.

P3. Reconstruction and signing.
- **a.** Reconstruct and sign $\mathtt{SignedBeaconBlock} \leftarrow \Pi_{rec}(w)$.

Fig. 4: DPaaS's runtime protocol Π_{DPaaS}.

Intuitively, a DPaaS protocol must provide the following guarantees. First, the auction phase admits only valid bids; we enforce this using BTs that validate contents before acceptance by PEs. Second, the winner finalization ensures outcome agreement for all honest PEs; this is provided by $\Pi_{\mathsf{Crab\text{-}BB}}$. Third, the outcome must be correct under auction semantics: the winner w is the highest bid in the globally active set $\mathcal{A}$, the latest subset of globally eligible bids defined by $\check{\mathcal{E}}$ and $\hat{\mathcal{E}}$. This last guarantee is not implied by agreement alone, because it requires the agreed output set to satisfy the correctness of the auction: bids that should be included by censorship resistance must remain represented, while bids in the output must be backed by enough replicas for availability. These are precisely the censorship-resistance and backed-availability guarantees provided by $\Pi_{\mathsf{Crab\text{-}BB}}$.

Based on the above observation, we provide Π_{DPaaS} in Fig. 4 and illustrate the protocol in the following.

B. Auction - P1.

In practice, the first bid for slot S usually arrives 2–4 seconds into slot $S-1$, as builders must wait for the prior block to propagate to construct a valid block. Unlike prior proposals [20] and prototypes [53], [58], our design keeps untrusted builder software outside the TCB: the BT only validates the final block, handles secret sharing of the payload, and attests to the bid information. Below, we discuss how our builder workflow differs from today's relay-based workflow.

PE Lookup and Verification. While relays have few, well-known endpoints that builders can use, DPaaS deployments and PEs are more decentralized. Each slot in an epoch

identifies the validator assigned as the proposer by their pk_V; these assignments are known one epoch (i.e., 6.4 minutes) in advance. The builder queries the metadata stored in the DPaaS registry contract (SC_R) to check if the pk_V is enrolled through DPaaS; if so, the builder retrieves the corresponding manifest M from the storage. If the configuration (e.g., providers, TEE implementations) meets the builder's trust assumptions, the builder connects to the PE endpoints in M via RA-TLS (similar to Sec. IV-B). If verification succeeds, the builder can submit their block to the PEs; however, the builder must interact with the BT to produce an attested bid for the PEs, which we describe next.

BT- Validation. The BT must first make sure the block is valid. To reduce the TCB of the BT, we do not run a full Ethereum node inside of the TEE; instead, the BT contains minimal libraries to support block simulation, relying on untrusted storage maintained by an Ethereum node running outside of the TEE. This decomposition is safe because the BT will include all information it cannot verify (e.g., the parent hash from untrusted storage) in the attestation (discussed next), such that the PEs can verify it with the Ethereum nodes that they necessarily must run to interact with other validators.

The BT first checks the builder's inputs for structural and contextual validity. Afterwards, the BT simulates transactions in the block, based on the most recent state maintained in the untrusted storage. The simulation attempts to sequentially execute the transactions in the block and computes the total payment value to the proposer. If any transactions fail to simulate, or the payment value is not sufficient compared to the bid, the BT returns an error.

BT- Data Preparation & Submission. Finally, BT prepares the necessary data for constructing the bid. Following the definition in Sec. VI-A, the bid consists of $(\mathtt{b}, bv, \mathtt{seq}, h, \mathtt{att}, \mathtt{spl})$. Among these fields, the BT extracts $(\mathtt{b}, bv, h)$ directly from the builder's block (after checks). The BT provides the `seq` by keeping track of previous bids in the slot, starting from $\mathtt{seq} = 0$. The BT constructs the `spl` where $\mathtt{pl} = (\mathtt{header}, \mathtt{txs})$ for all $i = 0..n-1$:

$$(\mathtt{esshr} : \{\mathtt{E}_{\mathtt{pk}^i_{\mathrm{asym}}}(K_i) \mid \mathtt{H}(K_i)\},\ \mathtt{header},\ \mathtt{E}_K(\mathtt{txs}),\ \mathtt{R}(\mathtt{pl}))$$

by first randomly generating a key K to encrypt the transactions as $\mathtt{E}_K(\mathtt{txs})$. The BT uses an $(f+1, n)-$secret sharing scheme to construct shares K_i where $i \in [0, n)$. For each K_i, the BT computes the hash $\mathtt{H}(K_i)$ and encrypts it for the PE_i $E_{pk^i_{\mathrm{asym}}}(K_i)$; the hash will be used by other PEs to verify decrypted shares during reconstruction (see Sec. VI-C). The `header`, including metadata (*e.g.*, `parent_hash`, `gas_limit`) of the block, will also be submitted. At this point, the BT generates a TEE attestation supplying the hash over all other elements of the bid including the block header as the user data field: $\mathtt{att} = \mathtt{att}\langle H(\mathtt{b}, bv, \mathtt{seq}, h, \mathtt{spl})\rangle$. The BT returns the bid to the builder software, which can send it to the PEs.

Bidding at PEs. Before T_e, each PE_i listens for bids from builders. Upon receiving a bid r, it performs several checks: 1) $r.\mathtt{seq}$ adheres to contiguous ordering from 0 for the builder, 2) verifies the $r.\mathtt{att}$ and signature over the carried user data, and 3) checks unverified information in the `header` from the BT against the current chain state (e.g., `slot`, `parent_hash`). If checks pass, PE_i adds r to E_i.

Moreover, DPaaS supports current top bid updates by enabling builders to subscribe to a stream from each PE_i, which publishes a message if r corresponds to the new highest bid in $\mathcal{E}_i$. Since we distinguish between local and global eligibility, the DPaaS builder library aggregates these messages to ensure that at least $f+1$ PEs agree on the top bid.

C. *Post-Auction - P2. & P3.*

After the auction deadline T_e, the PEs need to jointly determine the winning bid w from all of the locally eligible sets $\mathcal{E}_i$ (*finalization*), reconstruct the block payload from $w.\mathtt{spl}$ (*reconstruction*), and finally output an instance-level signature for the Signed Beacon Block (*interpolation*). We explain the procedures from high level here while leaving the detailed protocol to Appendix A.

Finalization. We instantiate $\Pi_{\mathsf{Crab\text{-}BB}}$ with the semantics that real builders as clients submit a stream of bids with continuous sequence number. Therefore, $\mathcal{E}_i[b]$ at honest PEs only contain contiguous sequence numbers from 0, each $\mathcal{V}_i$ need only to keep the highest-sequence bid, which implicitly captures earlier ones. We apply a summarization function AppCompress (Algorithm 2 in Appendix A) to only include the active bid for each builder. We also adapt the post-consensus function AppInterp (Algorithm 2) to operate on consensus output, ensuring that it preserves implicit inclusion.

Reconstruction. Given a winning bid w, PEs must now reconstruct the block payload by decrypting the `txs` from $w.\mathtt{spl}$. As shown in Fig. 9 in Appendix A, each PE_i starts by decrypting its own share using its private key and broadcasts the share to all PEs. When receiving a share, PE_i checks the validity of the share by comparing the hash of the K_j it receives the hash contained in $w.\mathtt{spl}$. Once PE_i receives valid shares from at least $f+1$ PEs, it can reconstruct the symmetric key K and decrypt the $\mathtt{E}_K(\mathtt{txs})$ contained in $w.\mathtt{spl}$ to get `pl`.

Interpolation. At least $f+1$ PEs obtain plaintext payload for the winning bid w and must produce a signed beacon block. Since the beacon block needs to be signed over all block fields, including `pl` and `cdata`, we include $\mathtt{R}(\mathtt{pl})$ in plaintext within $w.\mathtt{spl}$ instead of reconstructing `pl` first. This enables parallel reconstruction and interpolation, reducing latency. Each PE_i generates a partial signature σ_i and broadcasts it. After collecting $f+1$ valid partial signatures, PE_i uses Lagrange interpolation to compute the full signature (verifiable under pk_E) and propagates the Signed Beacon Block.

VII. Implementation

In this section, we describe our prototype implementation of DPaaS, which consists of several core components: BT and DPaaS library for builders, and the Control and PE applications for deployments. We implement the components in Rust, using Tonic [59] to support RPC-based communication

between them. For TEE components, we provide implementations for both AMD SEV-SNP [33] and Intel TDX [32]. We use Geth [60] and Lighthouse [61] as the Ethereum Node for PEs. We validate DPaaS end-to-end functionality through a private testnet supported by ethereum-package [62].

A. Builder TEE (BT)

We implement a single RPC, `validate_block`, which takes in a block (with raw transaction payload) and bid value from a builder. We use Reth [63] to simulate execution of all transactions against the beacon state for this slot; to restrict the TCB, we extract the minimal set of crates needed for simulation (not for full execution-node deployment). We compute the fee paid to the proposer by measuring the balance change to the address designated by the proposer to receive block rewards. We implement data preparation for the `spl` based on the open-source Shamir Secret Share library [64] with its Rust binding [65] and asymmetric encryption crate age [66]. For `spl`, we generate the common hash via SHA256 in the SHA-2 crate [67] and the Merkle root of the payload `R(pl)` via the tree_hash crate [68] for compatibility with Ethereum.

B. Proposer Entity (PE)

We implemented five RPC services for PEs that capture different phases of DPaaS: Bootstrap, Auction, Consensus, Recovery, and Signing. Below, we provide some details for these services.

Bootstrapping. We implemented a `mutual_attest` API for the PEs and a `verify` API for outside verification of the DPaaS instance by users and builders. Both of these use RA-TLS to support constructing a secure channel to the PEs and binding this channel to information contained in the attestation. At the time of our implementation, we could not find a Rust RA-TLS crate; therefore, we created our own implementation for AMD SEV-SNP and Intel TDX that build on top of the Rust bindings for OpenSSL [69] and handles the TEE-specific attestations.

Auction. We implemented a `submitBlock` API for builders and a `UpdateTopBid` API for anyone to use. For `submitBlock`, we leverage an in-memory cache to store the cryptographic keys for verifying TEE attestations. Whenever a PE detects a new builder public key, it queries the attestation service of the builder's hardware provider to obtain the corresponding verification key (*e.g.*, the VCEK for AMD SEV-SNP). We cache this key in memory, enabling faster verification for subsequent bids from the same builder.

Consensus. We implement $\Pi_{\text{Crab-BB}}$ with our application-specific compress and interp functions (see Algorithm 2). For messages relayed between PEs (e.g., signed `ex-fast-2` message), we use BLS signatures for authentication.

Recovery & Signing. To facilitate the fast recovery of the winning block, we implemented two APIs: `shareShare`, `shareBlsSignature`. We use the same secret sharing crate as with the BT implementation, and use blsttc library [70] to support threshold BLS signatures. We extend `shareBlsSignature` to support signing other messages beyond proposed blocks to support general validator duties (e.g., block attestation). We use asynchronous execution in Tokio [71] to run block reconstruction and BLS signature interpolation concurrently.

C. Limitations

Attested TEE Placement When selecting a DPaaS deployment, a user's policy may require that TEEs are spread across different availability zones. Currently, major cloud providers have endorsement keys [72] for TEE attestation tied to individual providers; however, these keys are global across all availability zones for a provider. In our implementation, we rely on services from providers (e.g., Instance Metadata Service for AWS [73]), which exist outside the TEEs, to learn this placement information. This limitation could be removed if providers move towards data center assurance in the future [74]. In the meantime, users can enforce their own deployment policy by selecting n distinct cloud providers as DPaaS deployments, avoiding dependence on untrusted metadata services.

VIII. Evaluation

A key goal of DPaaS is to deliver performance comparable to existing relays. In this section, we evaluate DPaaS across four dimensions:

- **BT Overhead**: Performance penalty introduced to builders as a result of the BT. (Sec. VIII-B)
- **Bid Processing Performance**: Latency from bid receipt to becoming locally eligible. (Sec. VIII-C)
- **Post-auction Performance**: Duration required for a winning bid to be finalized and incorporated into the final signed beacon block. (Sec. VIII-D)
- **Macrobenchmark**: Evaluation of timing for bid count and MEV when processing historical bid data. (Sec. VIII-E)

A. Dataset and Configuration

We use the full bid dataset from Ultra Sound Relay [75], which records the historical bids during 2023/09-2023/12. That's the newest public dataset as far as we know. After December 2023, Ultra Sound Relay stopped collecting full bid data due to privacy concerns. We randomly chose 40,000 slots during that time window for our analysis. Among the 40,000 slots, we compared them with the winning bids recorded on-chain [6] and got a winning-bid dataset with 28,107 winning bids. We also extended the winning-bid dataset to winning-builder dataset with 117,313 bids, which not only includes the winning bids but also includes all the winners' submission.

We deployed DPaaS on 4 CVMs across 4 availability zones provided by Amazon AWS, Microsoft Azure and Google Cloud Platform (GCP). Fig. 5 shows the round trip times measured between different CVMs. Each CVM has 4 vCPUs and 16 GB of memory. We deployed a BT in one CVM in AWS, with 8 vCPU and 32 GB memory.

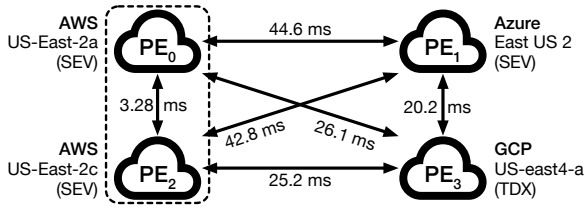

Fig. 5: DPaaS deployment topology for our evaluation ($n = 4$), labeled by RTTs (including signing and verification overhead).

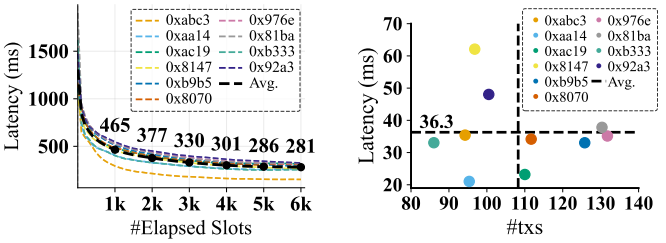

Fig. 6: Analysis of validation latency across 10 builders, labeled by public key prefix, and 6,000 slots. (a) Latency for the first block in each slot. (b) Scatter plot of "warmed" latency for subsequent blocks in each slot versus number of transactions, averaging over all slots.

For this cloud deployment, there are two changes that differ from our design and testnet deployment. First, we deploy the Ethereum archive node within the same CVM as the BT; however, our BT implementation still treats the data as untrusted. Second, for PEs, we simulate an archive node by extracting the necessary information for each slot (e.g., slot number, parent hash). The primary cost of checking block header information in received bids against the current chain state (e.g., parent hash) relates to querying the information from the live Ethereum node, which does not fit our evaluation of historical traces for comparison with relays (and only happens once per slot). Therefore, we exclude these checks entirely from per-bid latency measurements in Sec. VIII-C.

B. BT *Overhead*

BT could add overhead to builder's bidding due to block validation, block package data preparation, and TEE attestation generation. To evaluate this overhead, we simulate builder submissions using the extended winner-bid dataset, which has on average 108 transactions and payload size of 57.19 KB. Table I summarizes the detailed latency breakdown observed in BT. Data preparation and TEE attestation generation are lightweight operations: the former is a function of the payload size, while the latter is constant. Block validation, taking 53.1 ms on average, is the most significant because it requires simulating the execution of all transactions in the block. We derive this average over 28,107 slots in a single BT; however, we observed significant variance, which we explore next.

Block Validation Details. We observe that the first block validation in a slot incurs higher "cold-start" latency due to warming the in-memory EVM state cache. To quantify the builder-side impact, we sampled 10 active builders (i.e., bids in $>$ 50% slots) and collected their bids across 6,000 slots.

Fig. 6a shows the cold-start latency per slot: the first block a builder validates can exceed 1.5 s, but this cost drops quickly as the builder continues validating blocks. After 6,000 slots, the average cold-start latency falls to 281 ms. Importantly, these results reflect only the cold-start cost; all subsequent validations in the same slot benefit from a warmed cache. As shown in Fig. 6b, the average warmed validation latency across the 10 builders is just 36.3 ms.

This block-validation measurement shows that although cold-start latency is high, it can be effectively mitigated in two ways. First, running BT into auction for sufficient slots naturally amortizes the cost—after approximately 20 hours (6,000 slots), the overhead decreases by more than 78%. Second, a builder can proactively warm the in-memory cache each slot as it iteratively constructs the block.

Block Validation	Data Preparation	TEE Attestation	Total
53.12 ± 13.51	6.16 ± 1.13	4.92 ± 0.17	64.20 ± 18.3

TABLE I: BT latency (ms) breakdown.

C. *Bid Processing Performance*

We evaluate PE bid processing from the perspective of a single PE_i, from receiving a bid to placing it in the locally eligible set. We analyze the latency in an unloaded setting and then look at throughput saturation under load (as well as scalability with CPU resources). In both experiments, we use bids from the winning-bid dataset.

Unloaded. We set up a single builder that periodically submits bids in one-second intervals to a single PE_i. The results are in Table II, broken down into key components. Average overall bid processing latency is 4.83ms, which is significantly lower than relays as a result of TEE attestations removing the need to simulate the block transactions [76]. Most of this comes from signature verification (*Sig.*) for the TEE attestation (2.25ms) as well as the RPC framework processing (*Queue+Deserialize*). Performing sanity checks and verifying integrity through TEE attestation (*Int. Check*) as well as verifying the TCB (*TCB*) take little time.

Loaded. We deploy 20 builder threads on a single host, each periodically submitting bids to one PE_i to match target throughputs from 25 to 600 requests per second (step size 25). To assess DPaaS 's scalability, we vary the CPU allocation from 1 to 4 cores. For reference, we also plot the distribution of real-world bid arrival rates from the historical full-bid dataset. Bid submissions follow a steady-then-burst pattern: a moderate rate ($\leq$100 bids/s) for the first 9–10 seconds, followed by a sharp surge as builders race in the final seconds. Our analysis focuses on the peak-load region using 100 ms windows.

We present the results in Figure 7. The saturation points for 1, 2, and 4 cores are 150, 300, and 575 (respectively).

Queue+ Deserialize	Int. Check	Verify Attestation		Overall
		TCB	Sig.	
1.48±0.03	0.84±0.54	0.12±0.07	2.25±1.91	4.83±1.26

TABLE II: PE latency (ms) breakdown in unloaded setting.

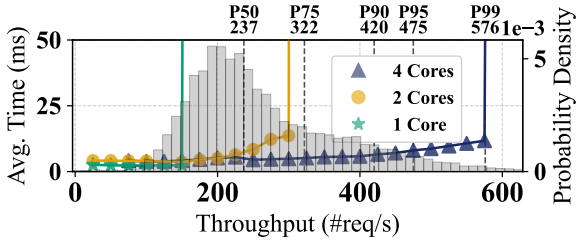

Fig. 7: Analyzing saturation via average processing latency versus bid throughput while varying CPU cores. PDF of real-world peak throughput from full-bid dataset.

This means that, with just 4 cores, a PE can handle $\geq 99\%$ of historical windows within $\leq$ 10 ms per request. We emphasize that this represents a stress test using historical peak throughput under continuous submission. In practice, the load typically follows burst pattern at around $t=-4$s to t=0s. Interestingly, we observed that existing relays cannot sustain such bursty workloads under limited resources, motivating optimistic relays [76] to mitigate critical-path latency.

D. Post-Auction Performance

Post-auction latency includes agreement on the winning bid, recovering the plaintext block payload, and signing the block. We generated eligible bid sets by randomly selecting 100 slots in 2023/12. We evaluate the latency in three different cases: 1) optimistic fast path, 2) fallback path with one follower crash, and 3) fallback path with first leader crash. We then input the 100 eligible bid sets to all n PEs under different cases. We explore all combinations of PEs for the initial leader and (if necessary) faulty party, running 100 trials per combination for each input bid set, to account for the differences in inter-PE RTTs (see Fig. 5). We set Δ to the 99th percentile of one-way latency measured over 10,000 samples between the PEs.

We present our results in Table III. All measured values can be bounded by functions of Δ, though they are not directly comparable to it, since they are based on the actual (and unknown) latency $\delta < \Delta$. For instance, in some rounds PE can proceed without hearing from all n PEs, which means we don't need to wait for slower nodes.

In Case 1, the leader can construct SQ having heard from all 4 PEs, leading to a three-round latency. In Case 2, due to a follower crash, the leader must use the fallback path and will wait for at most 2Δ prior to the VBB protocol; therefore, the upper bound on consensus in this case is 4Δ. In Case 3, the first leader crashed, so there will be a view change after a timeout of 4Δ. After the view change, it's guaranteed that new leader is honest and 2 more rounds are needed to commit. Therefore, the upper bound on consensus in this case is $4\Delta + 2\Delta + 2\Delta = 8\Delta$. Our implementation

Case	Consensus	Recovery		Total
		Shares	Signing	
Case 1	41.38	13.85	14.31	55.69
Case 2	79.57	16.55	18.80	94.27
Case 3	153.92	16.21	18.41	172.33

TABLE III: Average latency (ms) breakdown for finalization protocol. Shares and Signing take place in parallel, so Total only considers the highest in addition to Consensus.

exploits the independence between recovering the secret from shares and signing the block header to parallelize these tasks; therefore, the total is equivalent to the consensus latency plus the maximum of either shares or signing latencies. Finally, we include a more-detailed, per-combination breakdown for the consensus latency in Appendix E.

E. Macrobenchmark

We randomly selected 3,000 slots in 2023/12 and replayed all the bids during those slots to our DPaaS instance. Due to limited data availability regarding round-trip times between relays and builders in the builder market, we replayed bids based solely on the `ReceivedAt` timestamps in the dataset.

Bid Processing Performance For each slot, we tracked the number of eligible and active (non-canceled eligible) bids throughout the slot for both DPaaS and the relay; for DPaaS, we used globally eligible and active bids to provide an equivalent comparison. We also kept track of upper bounds for each of these: the number of received bids at any time bounds the number of eligible bids, while the number of unique builders seen bounds the number of active bids. We found that DPaaS significantly improves both of these counts due to lightweight bid validation in the PEs; meanwhile, relays need to rely on optimistic mode (i.e., marking bids for select builders as eligible without validation) and selectively prioritizing other non-optimistic bids. As an example, when the receiving a median of 1,403 bids near the end of the auction in total, DPaaS attains 1,402 eligible bids, whereas the relay yields only 214. We refer readers to the detailed results in Appendix F.

Winner Bid Value However, higher volume of eligible bids does not necessarily imply higher MEV values. Therefore, we analyzed the winning bid values in both DPaaS and the relay under different target block-proposal times: for DPaaS we used the user-configured T_p, while for the relay we consider the time `getHeader` with an offset of +880 ms to account for the average time before the `getPayload` (measured across 5,063 slots from the Ultra Sound Relay dataset). To match DPaaS 's use case, where users specify an upper bound on post-auction latency relative to T_p to determine T_e, we evaluated both the best case (4Δ) and worst case (9Δ since $f = 1$).

We present our results in Figure 8. At $t = 0$ s, which is the Ethereum-specified time for honest block proposal, DPaaS yields 0.0777 ETH (best case) and 0.0775 ETH (worst case), compared to 0.0752 ETH for the relay ($\geq 3\%$ improvements).

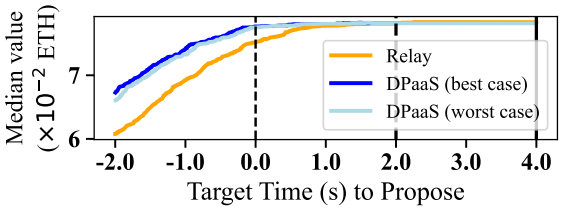

Fig. 8: Median winning bid value based on different target proposal times (T_p) for DPaaS versus Relays.

Although both DPaaS and the relay converge after $t = 2$ s when no new bids arrive, such delayed proposals risk not receiving sufficient attestations [10] leading to a missed slot. The negligible gap between best-case and worst case latency bounds further shows DPaaS 's stable profit capture.

Due to the variability of actual network latency in practice relative to our setting of Δ, it is possible that DPaaS may infrequently miss the target proposal time (which would not be captured in the plot). In this experiment, DPaaS proposed the block ahead of T_p for all 3,000 slots.

IX. Discussion

Costs and Incentives. Our design of DPaaS introduces some additional costs for builders and enrolled users. Builders incur additional costs due to the BT, which they must use to submit their bids for any DPaaS proposer slot. In terms of latency cost, our evaluation shows that this is modest at around 50 ms, and could be further improved via prefetching to warm up the cache while the builder constructs the block. In terms of monetary cost, the configuration we use in our evaluation costs 0.36 USD/hr; however, we note that builders can trade off latency and monetary cost (e.g., fewer vCPUs). Users who enroll as validators through DPaaS must pay the operator who manages the deployment. DPaaS supports multiple users (i.e., instances) for a single deployment; however, even if we consider the worst case of a single user, the configuration in our evaluation costs 0.66 USD/hr, which is around 2% of validator revenue (around 30 USD/hr [77]).

We believe these costs are justified by the economic nature of MEV extraction. Builders require payload privacy to protect their strategy until the block is committed to prevent MEV stealing attacks, which is especially important for exclusive order flows and high-value MEV opportunities. Additionally, users who enroll through DPaaS as validators can gain access to MEV opportunities, given that such builders may prefer to submit such blocks through DPaaS for stronger security guarantees. We look forward to deploying DPaaS, which will allow us to measure and analyze the economic effects.

Centralization. We distinguish design-level trust from deployment-level centralization. At the design level, DPaaS replaces relay trust with a threshold assumption over PEs: as long as at most f are faulty, guarantees hold. Each instance can publish its PE composition and attestation metadata, enabling verification across operators, regions, and TEE vendors, reducing opacity and single points of failure.

At the deployment level, centralization depends on adoption—e.g., validator concentration or dominant PE operators—and must be evaluated empirically. We expect DPaaS to increase choices and promote decentralization.

X. Related Work

Replacing Relays. TEE-Boost [20] is the first proposal to eliminate the relay from the block-building pipeline using a TEE to validate blocks, but does not address data availability. Based on TEE-Boost, Paradigm [19] proposed to leverage a validator committee running threshold cryptography [22] to address data availability; however, this involved consensus layer changes, which makes it hard to deploy and is mismatched with dynamic availability goals in Ethereum. In contrast, DPaaS provides a backward compatible solution while addressing all fair exchange problems.

Enshrined PBS (ePBS) [18], currently under development, proposes to eliminate relays by staking builders to support fair exchange. This requirement disadvantages small builders, who are likely to incur negative profits if they must lock up 32 ETH. Additionally, ePBS places all interactions into the p2p network, which could lead to loss of MEV due to higher latency. Third, ePBS operates as a single-bid open auction, which restricts builder bidding strategies (e.g., using bid cancellations). Since ePBS does not prevent sidecar protocols, DPaaS remains complementary and may be preferred by validators and (small) builders.

Censorship-Resistant Byzantine Broadcast. There are two concurrent works that formalize definitions similar to Crab-BB. First, Abraham *et al.* [78], [79] formalize censorship-resistant Byzantine broadcast (CRBB), a closely related problem that also seeks to prevent consensus from ignoring sufficiently disseminated client inputs. However, CRBB does not solve the backed-availability challenges faced by DPaaS. They present a lower bound on the latency required to obtain this property; our protocol matches this lower bound while also obtaining backed availability. Another concurrent work, Multiple Concurrent Proposers (MCP) [80], also targets censorship-resistant broadcast and achieves backed availability while matching the CRBB lower bound. However, MCP is designed for larger validator committee with Byzantine-fraction smaller than $1/5$. Their security guarantees can't hold under $1/4$ (*i.e.*, the most practical setting $n = 4, f = 1$ in DPaaS). We added one round in $\Pi_{\text{Crab-BB}}$ and fill this gap. Moreover, we show we can remove the second exchange when $f > 1$ or $n > 4f$ to meet CRBB's lower bound in Appendix E.

Decentralized Auction. Previous protocols [81], [82], [83], [84], [85], [86] focus on decentralizing sealed-bid auction. In contrast, we focus on an open-bid, sealed-payload decentralized auction while supporting bid-cancellation. However, previous works either treat bid cancellation as out of scope [81], [82], [84], [85], [86] or malicious behavior [83].

XI. Conclusion

Proposer-Builder Separation (PBS) is vital to Ethereum's goal of decentralization, yet the current MEV-Boost architecture introduces centralization and security risks through its reliance on trusted relays. In this paper, we presented Decentralized Proposer-as-a-Service (DPaaS), which distributes the combined roles of the proposer and relay across a set of Proposer Entities (PEs). We developed fault-tolerant protocols spanning the offline, auction, and post-auction phases, meeting fair-exchange goals while maintaining compatibility with Ethereum. Our evaluation demonstrates that DPaaS delivers strong performance, scalability, and ability to capture MEV in comparison to relays.

Acknowledgements

This work is supported by Flashbots MEV Fellowship Grants. We are grateful to Ultra Sound Relay and Sen Yang for sharing the full-bid dataset and providing access to relay-timestamp information. We also appreciate the valuable discussions with Jonathan Passerat-Palmbach and Quintus Kilbourn.

References

[1] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability," in *IEEE Symposium on Security and Privacy*, 2020.

[2] Ethereum Foundation, "Maximal extractable value (MEV)," August 2025. [Online]. Available: http://ethereum.org/developers/docs/mev/

[3] V. Buterin, "Proposer/block builder separation-friendly fee market designs," June 2021. [Online]. Available: https://ethresear.ch/t/proposer-block-builder-separation-friendly-fee-market-designs/9725

[4] M. Bahrani, P. Garimidi, and T. Roughgarden, "Centralization in block-building and proposer-builder separation," in *Financial Cryptography and Data Security (FC)*, 2024.

[5] Ethereum Foundation, "Proposer-Builder Separation," https://ethereum.org/en/roadmap/pbs/, 2024, accessed: 2025-3-30.

[6] S. Yang, K. Nayak, and F. Zhang, "Decentralization of Ethereum's Builder Market," in *IEEE Symposium on Security and Privacy*, 2025.

[7] EspressoSystems, "Fair Exchange in Proposer-Builder Separation," October 2023. [Online]. Available: https://hackmd.io/@EspressoSystems/fair-exchange-in-proposer-builder-separation

[8] B. Adams and D. Feist, "EIP-7782: Reduce Block Latency," October 2024, ethereum Improvement Proposals, no. 7782. [Online]. Available: https://eips.ethereum.org/EIPS/eip-7782

[9] F. Wu, T. Thiery, S. Leonardos, and C. Ventre, "Strategic bidding wars in on-chain auctions," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2024.

[10] C. Schwarz-Schilling, F. Saleh, T. Thiery, J. Pan, N. Shah, and B. Monnot, "Time Is Money: Strategic Timing Games in Proof-Of-Stake Protocols," in *Conference on Advances in Financial Technologies, AFT*, 2023.

[11] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, and A. Bakhta, "EIP-1559: Fee market change for ETH 1.0 chain," April 2019, ethereum Improvement Proposals, no. 1559. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1559

[12] M. Kalinin, D. Ryan, and V. Buterin, "EIP-3675: Upgrade consensus to Proof-of-Stake," July 2021, ethereum Improvement Proposals, no. 3675. [Online]. Available: https://eips.ethereum.org/EIPS/eip-3675

[13] M. Peterson, "Ethereum consensus shifts on EVM upgrade," https://blockworks.co/news/ethereum-consensus-evm-upgrade-fusaka-devs, 2025, accessed: 2025-10-20.

[14] mikeneuder, "Bid cancellations considered harmful," https://ethresear.ch/t/bid-cancellations-considered-harmful/15500, 2023.

[15] Flashbots, "MEV-Boost Overview," https://docs.flashbots.net/flashbots-mev-boost/introduction, 2024.

[16] Flashbots, "Post mortem: April 3rd." [Online]. Available: https://collective.flashbots.net/t/post-mortem-april-3rd-2023-mev-boost-relay-incident-and-related-timing-issue/1540

[17] T. Wahrstätter, "MEV-Boost Dashboard," https://mevboost.pics/, 2025, accessed: 2025-3-30.

[18] F. D'Amato, B. Monnot, M. Neuder, Potuz, and T. Tsao, "EIP-7732: Enshrined Proposer-Builder separation [DRAFT]," https://eips.ethereum.org/EIPS/eip-7732, June 2024, Ethereum Improvement Proposals.

[19] Paradigm, "Removing the Relays," https://www.paradigm.xyz/2024/10/removing-the-relays, October 2024.

[20] Flashbots, "TEE-Boost," https://collective.flashbots.net/t/tee-boost/3741, August 2024.

[21] Flashbots, "Mev-boost risks and considerations," April 2025. [Online]. Available: https://docs.flashbots.net/flashbots-mev-boost/architecture-overview/risks

[22] S. Garg, D. Kolonelos, G.-V. Policharla, and M. Wang, "Threshold Encryption with Silent Setup," Cryptology ePrint Archive, Paper 2024/263, 2024. [Online]. Available: https://eprint.iacr.org/2024/263

[23] B. Vedartham, "Proposer Relay Interactions with ePBS," https://hackmd.io/@bchain/BJYoUYjIll, 2025, accessed: 2025-10-20.

[24] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, and D. Song, "Ekiden: A Platform for Confidentiality-Preserving, Trustworthy, and Performant Smart Contracts," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.

[25] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the presence of partial synchrony," *Journal of the ACM (JACM)*, vol. 35, no. 2, pp. 288–323, 1988.

[26] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, "Combining GHOST and casper," *arXiv preprint arXiv:2003.03052*, 2020.

[27] Ethereum Foundation, "How to stake your ETH," https://ethereum.org/en/staking/, 2024, accessed: 2025-3-30.

[28] ——, "The Beacon Chain," https://ethereum.org/roadmap/beacon-chain/, 2025, accessed: 2025-11-12.

[29] G. Konstantopoulos and M. Neuder, "Time, slots, and the ordering of events in Ethereum Proof-of-Stake," https://www.paradigm.xyz/2023/04/mev-boost-ethereum-consensus, April 2023.

[30] T. Thiery, "Empirical analysis of Builders' Behavioral Profiles (BBPs)," https://ethresear.ch/t/empirical-analysis-of-builders-behavioral-profiles-bbps/16327, Augest 2023.

[31] V. Costan and S. Devadas, "Intel SGX explained," *IACR Cryptol. ePrint Arch.*, p. 86, 2016.

[32] Intel, "Intel trust domain extensions whitepaper," https://www.intel.com/content/dam/develop/external/us/en/documents/tdx-whitepaper-final9-17.pdf, 2020, accessed: 2025-3-30.

[33] D. Kaplan, J. Powell, and T. Woller, "AMD SEV-SNP: Strengthening vm isolationwith integrity protection and more," *White paper, Advanced Micro Devices Inc*, 2020.

[34] G. Chen, S. Chen, Y. Xiao, Y. Zhang, Z. Lin, and T. Lai, "SgxPectre: Stealing Intel Secrets from SGX Enclaves Via Speculative Execution," in *IEEE European Symposium on Security and Privacy*, 2019.

[35] S. van Schaik, A. Kwong, D. Genkin, and Y. Yarom, "SGAxe: How SGX fails in practice," https://sgaxe.com/, 2020.

[36] B. Schlüter, C. Wech, and S. Shinde, "Heracles: Chosen plaintext attack on amd sev-snp," in *ACM Conference on Computer and Communications Security (CCS)*, 2025.

[37] S. Gast, H. Weissteiner, R. L. Schröder, and D. Gruss, "Counterseveillance: Performance-counter attacks on AMD SEV-SNP," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[38] A. Shamir, "How to Share a Secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[39] G. R. Blakley, "Safeguarding cryptographic keys," in *International Workshop on Managing Requirements Knowledge, MARK*, 1979, pp. 313–318.

[40] J. von zur Gathen and J. Gerhard, "Fast polynomial evaluation and interpolation," in *Modern Computer Algebra*, 3rd ed. Cambridge, UK: Cambridge University Press, 2013, ch. 10.

[41] D. Boneh, B. Lynn, and H. Shacham, "Short Signatures from the Weil Pairing," in *Advances in Cryptology - ASIACRYPT International Conference on the Theory and Application of Cryptology and Information Security*, 2001.

[42] J. Drake, "Pragmatic signature aggregation with BLS," https://ethresear.ch/t/pragmatic-signature-aggregation-with-bls/2105?u=benjaminion, May 2018.

[43] D. Dolev and H. R. Strong, "Authenticated algorithms for byzantine agreement," *SIAM J. Comput.*, vol. 12, no. 4, pp. 656–666, 1983.

[44] I. Abraham, K. Nayak, L. Ren, and Z. Xiang, "Good-case Latency of Byzantine Broadcast: a Complete Categorization," in *ACM Symposium on Principles of Distributed Computing (PODC)*, 2021.

[45] I. Abraham, K. Nayak, L. Ren, and Z. Xiang, "Brief note: Fast authenticated byzantine consensus," *arXiv preprint arXiv:2102.07932*, 2021.

[46] Flashbots, "MEV-Boost Relay," October 2025. [Online]. Available: https://github.com/flashbots/mev-boost-relay

[47] AWS, "Precision clock and time synchronization on your EC2 instance." [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/set-time.html

[48] B. Schlüter, S. Sridhara, A. Bertschi, and S. Shinde, "Wesee: Using malicious #vc interrupts to break AMD SEV-SNP," in *IEEE Symposium on Security and Privacy*, 2024.

[49] D. Lee, D. Jung, I. T. Fang, C. Tsai, and R. A. Popa, "An off-chip attack on hardware enclaves via the memory bus," in *USENIX Security*, 2020.

[50] J. Chuang, A. Seto, N. Berrios, S. van Schaik, C. Garman, and D. Genkin, "Tee.fail: Breaking trusted execution environments via DDR5 memory bus interposition," in *IEEE Symposium on Security and Privacy*, 2026.

[51] A. Seto, O. K. Duran, S. Amer, J. Chuang, S. van Schaik, D. Genkin, and C. Garman, "Wiretap: Breaking server SGX via DRAM bus interposition," in *ACM Conference on Computer and Communications Security (CCS)*, 2025.

[52] J. De Meulemeester, D. Oswald, I. Verbauwhede, and J. Van Bulck, "Battering RAM: Low-cost interposer attacks on confidential computing via dynamic memory aliasings," in *IEEE Symposium on Security and Privacy*, 2026.

[53] Flashbots, "Introducing BuilderNet," https://buildernet.org/blog/introducing-buildernet, 2024.

[54] Ethereum, "Keys in proof-of-stake ethereum," https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/keys/, 2025, accessed: 2025-07-01.

[55] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme," in *Public Key Cryptography - PKC, 6th International Workshop on Theory and Practice in Public Key Cryptography*, 2003.

[56] T. P. Pedersen, "A threshold cryptosystem without a trusted party (extended abstract)," in *Advances in Cryptology - EUROCRYPT, Workshop on the Theory and Application of of Cryptographic Techniques*, 1991.

[57] E. Syta, P. Jovanovic, E. Kokoris-Kogias, N. Gailly, L. Gasser, I. Khoffi, M. J. Fischer, and B. Ford, "Scalable bias-resistant distributed randomness," in *IEEE Symposium on Security and Privacy*, 2017.

[58] A. Lu, "Searching in TDX," https://collective.flashbots.net/t/searching-in-tdx/3902, 2024.

[59] Hyperium, "Tonic," https://github.com/hyperium/tonic, 2022.

[60] go-Ethereum, "Geth," May 2026. [Online]. Available: https://geth.ethereum.org/docs

[61] S. Prime, "Lighthouse: Ethereum consensus client," https://github.com/sigp/lighthouse, 2025, accessed: 2025-07-01.

[62] ethPandaOps, "Ethereum package," https://github.com/ethpandaops/ethereum-package, 2026, accessed: 2026-05-06.

[63] Paradigm, "Reth," August 2025. [Online]. Available: https://reth.rs/

[64] A. Sprenkels, "Shamir secret sharing library," October 2025. [Online]. Available: https://github.com/dsprenkels/sss

[65] ——, "Shamir secret sharing in Rust," October 2025. [Online]. Available: https://github.com/dsprenkels/sss-rs

[66] J. Grigg, "rage: Rust implementation of age," October 2025. [Online]. Available: https://github.com/str4d/rage

[67] Rust, "Crate sha2," October 2025. [Online]. Available: https://docs.rs/sha2/latest/sha2

[68] ——, "Crate tree_hash," October 2025. [Online]. Available: https://docs.rs/tree_hash/latest/tree_hash

[69] ——, "Crate openssl," https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/keys/, 2025, accessed: 2025-07-01.

[70] MaidSafe, "BLST Threshold Crypto (blsttc)," October 2025. [Online]. Available: https://github.com/maidsafe

[71] Tokio, "Tokio - An asynchronous Rust runtime," October 2025. [Online]. Available: https://tokio.rs/

[72] AMD, "Versioned Chip Endorsement Key (VCEK) Certificate and KDS Interface Specification," https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/specifications/57230.pdf.

[73] Amazon AWS, "Use instance metadata to manage your EC2 instance," October 2025. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-metadata.html

[74] F. Rezabek, M. Mahhouk, A. Miller, Q. Kilbourn, G. Carle, and J. Passerat-Palmbach, "Proof of cloud: Data center execution assurance for confidential vms," 2026. [Online]. Available: https://arxiv.org/abs/2510.12469

[75] Ultra Sound, "Ultra Sound Relay." [Online]. Available: https://relay.ultrasound.money/

[76] A. Chiplunkar and M. Neuder, "Optimistic relays and where to find them," https://frontier.tech/optimistic-relays-and-where-to-find-them, 2025, accessed: 2025-9-28.

[77] Etherscan, "Ethereum Charts & Statistics," 2026. [Online]. Available: https://etherscan.io/charts#section-blockchain-data

[78] I. Abraham, Y. Efron, and L. Ren, "The latency cost of censorship resistance," *IACR Cryptol. ePrint Arch.*, vol. 2025, p. 2136, 2025. [Online]. Available: https://eprint.iacr.org/2025/2136

[79] Abraham, Ittai and Efron, Yuval and Ren, Ling, "Latency cost of censorship resistance," https://decentralizedthoughts.github.io/2026-04-23-latency-of-censorship-resistance/, 2026, accessed: 2026-4-24.

[80] P. Garimidi, J. Neu, and M. Resnick, "Multiple concurrent proposers: Why and how," Cryptology ePrint Archive, Paper 2025/1772, 2025. [Online]. Available: https://eprint.iacr.org/2025/1772

[81] M. K. Franklin and M. K. Reiter, "The design and implementation of a secure auction service," in *IEEE Symposium on Security and Privacy*, 1995.

[82] S. Bag, F. Hao, S. F. Shahandashti, and I. G. Ray, "SEAL: Sealed-bid auction without auctioneers," Cryptology ePrint Archive, Paper 2019/1332, 2019. [Online]. Available: https://eprint.iacr.org/2019/1332

[83] N. Tyagi, A. Arun, C. Freitag, R. S. Wahby, J. Bonneau, and D. Mazières, "Riggs: Decentralized sealed-bid auctions," in *ACM Conference on Computer and Communications Security (CCS)*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds., 2023.

[84] N. Glaeser, I. A. Seres, M. Zhu, and J. Bonneau, "Cicada: A framework for private non-interactive on-chain auctions and voting," Cryptology ePrint Archive, Paper 2023/1473, 2023. [Online]. Available: https://eprint.iacr.org/2023/1473

[85] A. Novakovic, A. Kavousi, K. Gurkan, and P. Jovanovic, "Cryptobazaar: Private sealed-bid auctions at scale," *IACR Cryptol. ePrint Arch.*, p. 1410, 2024. [Online]. Available: https://eprint.iacr.org/2024/1410

[86] K. Zhong, Y. Ma, Y. Mao, and S. Angel, "Addax: A fast, private, and accountable ad exchange infrastructure," in *Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

[87] Ethereum, "Simpleserialize (ssz)," https://github.com/ethereum/consensus-specs/blob/v1.3.0/ssz/simple-serialize.md, 2021, accessed: 2025-07-01.

[88] A. Stokes, A. Dietrichs, D. Ryan, M. H. Swende, and lightclient, "EIP-4788: Beacon block root in the EVM," Febrary 2022, ethereum Improvement Proposals, no. 4788. [Online]. Available: https://eips.ethereum.org/EIPS/eip-4788

APPENDIX A
DETAILED SUB-PROTOCOL AND ALGORITHM FOR Π_{DPaaS}

The AppCompress and AppInterp is shown in Algorithm 2, which are the functions instantiated by DPaaS semantic for the input Compress and Interp to $\Pi_{\mathsf{Crab\text{-}BB}}$. AppCompress summarizes the $\mathcal{V}_i$ from $\mathcal{E}_i$ by extracting the most recent bid for each builder (Line 1,2). AppInterp takes the input Val then outputs a set of bids. If Val is a strong quorum (SQ), just extract all the summarization $\mathcal{V}_i$ into a set $\mathcal{C}$ (Line 4,5); otherwise, if Val is set of a weak quorum (WQs), it extracts all summarization $\mathcal{V}_i$ from unique PEs into a set $\mathcal{C}$ (Line 6-11). For every builder, a candidate bid multi-set will be formed (Line 14) by collecting all bids appeared explicitly in the summarizations $\mathcal{V}_i$ of $\mathcal{C}$. Then based on the multi-set just formed, the function selects each builder's $(2f+1)$-th bid in decreasing order of sequence number (i.e., bid with the largest sequence number supported by at least $2f+1$ PEs) and aggregates those bids into the global active set–$\mathcal{A}$ (Line 15).

We will show in Appendix C, this preserves the definition of implicit membership.

The protocol for Reconstruction and Interpolation phase is shown in Fig. 9. Each PE_i starts by decrypting its own share using its private key and broadcasts the share to all PEs (Step 1). When receiving a share, PE_i checks the validity of the share by comparing the hash of the K_j it receives the hash contained in $w.\texttt{spl}$ (Step 2). Once PE_i receives valid shares from at least $f+1$ PEs, it can reconstruct the symmetric key K and decrypt the encrypted transactions contained in $w.\texttt{spl}$. At last PE_i is able to recover the full payload (Step 3).

In Ethereum, the final signature of the beacon block should be over the root hash of all fields of the beacon block, including the payload `pl` and `cdata`. Instead of waiting to reconstruct the full payload with transactions in plaintext from $w.\texttt{spl}$ to compute $\texttt{R}(\texttt{pl})$, we instead optimize this by including $\texttt{R}(\texttt{pl})$ in plaintext as part of $w.\texttt{spl}$. After winner finalization, the root hash r of whole winning block can be directly computed given the root hash of the two necessary fields: consensus data and payload (Step 4). Therefore, we can execute the Reconstruction (Step 1-3) and Interpolation (Step 4-7) in parallel given the independency of the two procedure, which reduces the latency from end-auction to proposal.

For signature interpolation, each PE_i generates its partial signature σ_i and broadcasts this to all other PEs (Step 5). Meanwhile, when receiving any signatures from other PEs, the PE_i also verifies it against their public BLS partial public key generated during offline (Step 6). After receiving $f+1$ valid partial signatures, PE_i uses Lagrange interpolation to compute the full signature that can be verified by the aggregate public key pk_V (Step 7).

APPENDIX B
DETAILED PROOF OF PROPERTIES FOR $\Pi_{\mathsf{CRAB\text{-}BB}}$

First, we introduce the correctness of Π_{vbb}, which can be instantiated by $(5f-1)$-psync-VBB protocol as four lemmas, which have been proved [44], [45].

Input: winning bid w, consensus data `cdata`,
Output: `SignedBeaconBlock`

1. Decrypt the local payload-key share
$$K_i \leftarrow \mathcal{D}_{sk^i_{\text{asym}}}\big(w.\texttt{spl.esshr}[i]\big),$$
broadcast $\langle \texttt{share}, K_i\rangle_i$, and insert K_i into S.
2. Upon receiving $\langle \texttt{share}, K'_j\rangle_j$ from PE_j, insert K'_j into S only if $H(K'_j) = w.\texttt{spl.esshr}[j].h$.
3. Once $|S| \geq f+1$, recover the symmetric key and decrypt the winning block's transactions:
$$K \leftarrow \textsc{RecoverKey}(S).$$
$$\texttt{pl} \leftarrow \{w.\texttt{spl.header} \mid \mathcal{D}_K(w.\texttt{spl.E}_K(\texttt{txs}))\}.$$
4. Compute the signing root using the consensus data and the payload root committed in the sealed payload:
$$r \leftarrow \mathcal{R}\big(\mathcal{R}(\texttt{cdata}), w.\texttt{spl}.\mathcal{R}(\texttt{pl})\big).$$
5. Produce a partial BLS signature $\sigma_i \leftarrow \textsc{Sign}_{sk^i_{\text{p\_sign}}}(r)$. Then broadcast $\langle \texttt{psig}, \sigma_i\rangle_i$ and insert σ_i into I.
6. Upon receiving $\langle \texttt{psig}, \sigma_j\rangle_j$ from PE_j, insert σ_j into I only if
$$\textsc{Verify}_{pk^j_{\text{p\_sign}}}(\sigma_j, r) = \texttt{true}.$$
7. Once $|I| \geq f+1$, interpolate the final BLS signature $\sigma \leftarrow \textsc{AggregateSig}(I)$, and output $\texttt{SignedBeaconBlock} = (\texttt{pl}, \sigma)$.

Fig. 9: Protocol Π_{rec} for reconstructing the winning payload and producing the threshold BLS signature.

Algorithm 2 DPaaS Compress and Interp Functions

```
1:  function AppCompress(E_i)
2:      return V_i = ⋃_{b∈B} arg max_{e∈E_i[b]}(e.seq)
3:  function AppInterp(Val)
4:      if Val is SQ then
5:          C ← {Val}
6:      else
7:          C ← ∅
8:          procPE ← n * [false]
9:          for WQ_i ∈ Val, V_j ∈ WQ_i do ▷ i ordered deterministicly
10:             if procPE[j] == false then
11:                 C = C ∪ {V_j}; procPE[j] = true
12:     A ← ∅
13:     for all b ∈ B do
14:         cand[b] ← [ V_i[b] | V_i ∈ C, b ∈ dom(V_i) ]   ▷ multi-set
15:         A[b] ← ord↓_{2f+1}(cand[b]; e ↦ e.seq)
16:     return A
```

Lemma 1. ***Agreement****: If an honest replica commits* Val*, no honest replica commits any* $\text{Val}' \neq \text{Val}$ *in* Π_{vbb}*.*

Lemma 2. ***Termination****: Every replica eventually commits and terminates in* Π_{vbb}*.*

Lemma 3. ***Validity****: In* Π_{vbb}*, if the designated broadcaster replica is honest and* $\text{GST} = 0$*, then all honest replicas commit the broadcaster's value; otherwise the value* v *committed by any honest replica satisfies* $\mathcal{F}(v) = \texttt{true}$ *(*v *is externally valid).*

Algorithm 3 External validity used by Π_{vbb}

Input: Val. **Output:** `True` or `False`

function $\mathcal{F}$(Val)
  if Val $= \perp$ **then**
    return `False`
  if all relayed message signatures in Val verified **then**
    if $n = 4$ **then**
      if Val contains either one SQ or $f+1$ WQ **then**
        return `True`
    else if $n > 4$ **then**
      if Val contains one WQ **then**
        return `True`
  return `False`

Lemma 4. ***Good-case latency****: When* GST $= 0$ *and the leader is honest, the proposal of the leader will be committed within 2 rounds in* Π_{vbb}.

Given all the lemmas guarantees the correctness of Π_{vbb}, we can prove that the $\Pi_{\text{Crab-BB}}$ solves the Crab-BB problem as defined in Definition 1. We start from basic Agreement and Termination (Theorem 1), then Non-triviality (Theorem 2), at last censorship-resistance and availability-backed (Theorem 3).

Theorem 1. *If* Π_{vbb} *achieves Agreement and Termination,* $\Pi_{\text{Crab-BB}}$ *(Fig. 3) achieves Agreement and Termination as well: if one honest replica outputs* W*, no other honest replicas decide on output as* $W' \neq W$*; and all honest replicas eventually decide a value.*

Proof. For Agreement, by Lemma 1, honest replicas do not disagree on the output of evidence value Val from Π_{vbb}. After applying the same deterministic function Interp, honest replicas do not disagree on the output W. For Termination, after GST, Steps 2 and 3 in Fig. 3 can be completed by 2 rounds at most, which requires 2Δ time after which all honest replicas enter Π_{vbb}. By Lemma 2, all honest replicas decide and terminate. □

Theorem 2. *Assume* Π_{vbb} *is correct. Then* $\Pi_{\text{Crab-BB}}$ *satisfies Non-triviality against any* f*-bounded adversary when* $n \geq 5f - 1$ *with external validity function* $\mathcal{F}$ *defined as in Algorithm 3.*

Proof. Observe that Π_{vbb} requires parties to input externally valid values as defined in Algorithm 3. Moreover, Steps 2 and 3 ensure that honest parties enter Π_{vbb} only after receiving either one SQ or $f+1$ WQ. Any Val outside of this scope will result in $\mathcal{F}(\text{Val}) = \texttt{false}$ and never become committed by Π_{vbb}.

□

Theorem 3. *Assume* Π_{vbb} *is correct. Then* $\Pi_{\text{Crab-BB}}$ *with deterministic output interpretation satisfies censorship-resistance and availability-backed against any* f*-bounded adversary when* $n = 5f - 1$.

Proof. We first consider censorship resistance. Assume GST $= 0$, and let u be an input broadcast by client c_j. Suppose that every honest replica P_i includes u in its local input set, i.e., $u \in E_i$. Then, by Theorem 2, the output must have one of two forms: either it is certified by a SQ, or it is certified by $f+1$ WQ values.

In case (i), inside Val $= SQ$, the number of distinct input set $|SQ| \geq n - f + 1 = 4f$. Among the SQ, there should exist $4f - f = 3f$ honest replicas' input set E_i. That implies, if an input u is made to all honest replicas, it must appear in at least $3f \geq 2f + 1$ sets in SQ, by the rule in Step 5 of Fig. 3, the client input $u \dot{\in} W$.

In case (ii), after GST, within Δ, all honest replicas' `ex-1` messages can be delivered. Therefore all honest replica's should include at least $n - f$ distinct validly signed `ex-1` messages to their either `ex-fast-2` or `ex-normal-2` messages to be broadcast at $T = \Delta$. When the output is Val $= WQ$, that implies the the leader driving commit proposes a WQ which contain $f+1$ distinct validly signed `ex-normal-2` messages. Among these `ex-normal-2` messages, there must be one `ex-normal-2` from an honest replica, which must include all honest replica's `ex-1` messages carrying their input set. Then the argument follows case (i), if a client input u appear in all honest replicas' input set, it must exist in every honest replica's `ex-1` messages. By $n - f \geq 2f + 1$ and the rule in Step 5 of Fig. 3, the client input $u \dot{\in} W$.

Then we focus on availability-backed. Once a client input u appears in the output (*i.e.*, $u \dot{\in} W$), by the rule in Step 5 of Fig. 3, $u \in \text{Supp}_{2f+1}(\text{Val})$ which implies there must exist at least $2f + 1$ replicas' compressed input set implicitly containing such input u, among which at most f replicas are Byzantine. Therefore, at least $f + 1$ honest replicas can back such input implicitly containing such input u to exchange and explicitly backing such input u, the claim is proved. □

Appendix C
Detailed Proof of Properties in Application-level for Π_{DPaaS}

The correctness of Π_{DPaaS} relies on three parts: (1) the f-bounded adversary among the heterogeneous TEEs; (2) the threshold BLS-signature and threshold secret share scheme are correct; (3) the correctness of $\Pi_{\text{Crab-BB}}$; (4) the input compression and output interpretation accurately falls into the implicit inclusion. As (1) is the basic assumption justified in our threat model Sec. III-C, (2) has been proved in the cryptographic literature, (3) has been proved in Appendix B, we prove mainly (4) to conclude the correctness of Π_{DPaaS}.

Theorem 4. *Failed Trade Privacy (Goal 4, Def. 3.5) always holds in the protocol* Π_{DPaaS}.

Proof. According to Theorem 1, at least $n - f$ honest PEs will decide on the same winning bid w and only share the winning block's secret share. For all other bids, no honest PE will release their secret share; moreover, since the reconstruction

threshold is $f+1$, the $\leq f$ faulty PEs cannot recover these bids. □

Remark: Theorem 4 guarantees that all honest replicas will help recover the only winning bid; the losing bids cannot be recovered. In contrast, any centralized escrow like MEV-Boost relay does not have such a guarantee.

For rest of properties, we need to first define the semantic of bidding in the PBS setting and then prove the input/output wrapper in Π_{DPaaS} (*i.e.*, Algorithm 2) is correct.

Definition 4. *Bid Order: For each builder* $\mathtt{b} \in \mathcal{B}$*, let* $E_i[\mathtt{b}]$ *denote the set of bids from* $\mathtt{b}$ *contained in replica* PE_i*'s local input set* E_i*. For two bids* $e, e' \in E_i[\mathtt{b}]$*, we define the builder-local order*

$$e \preceq_{\mathtt{b}} e' \quad \Longleftrightarrow \quad e.\mathtt{seq} \leq e'.\mathtt{seq}.$$

Among bids from the same builder, the bid with the larger sequence number is considered more recent.

Definition 5. *DPaaS representation relation: Let* V *be an active-bid summary. For a bid* e*, we write*

$$e \overset{\bullet}{\in} V$$

if and only if V *contains a bid* $\hat{e}$ *from the same builder such that* $e \preceq_{\mathtt{b}} \hat{e}$*:*

$$e \overset{\bullet}{\in} V \quad \Longleftrightarrow \quad \exists \hat{e} \in V : \hat{e}.\mathtt{b} = e.\mathtt{b} \ \wedge \ e \preceq_{\mathtt{b}} \hat{e}.$$

Thus, an active-bid summary explicitly represents the bids it stores and implicitly represents all earlier bids from the same builder.

Lemma 5. *Compression soundness: For every local eligible bid set* $\mathcal{E}_i$*, let*

$$\mathcal{V}_i = \mathsf{AppCompress}(\mathcal{E}_i).$$

Then for every bid e*,*

$$e \overset{\bullet}{\in} \mathcal{V}_i \quad \Longleftrightarrow \quad e \in \mathcal{E}_i.$$

Proof. Fix a builder $\mathtt{b}$. If $\mathcal{E}_i[\mathtt{b}] = \emptyset$, then $\mathsf{AppCompress}(\mathcal{E}_i)$ contains no bid from $\mathtt{b}$. Therefore, by Definition 5, no bid from $\mathtt{b}$ is represented by $\mathcal{V}_i$. Hence, for every bid e from $\mathtt{b}$, both $e \overset{\bullet}{\in} \mathcal{V}_i$ and $e \in \mathcal{E}_i$ are false.

Otherwise, let

$$e^\star = \underset{e \in \mathcal{E}_i[\mathtt{b}]}{\arg\max}\, e.\mathtt{seq}.$$

By definition of AppCompress, the summary $\mathcal{V}_i$ contains exactly $e^\star$ for builder $\mathtt{b}$.

We first show the forward direction. Suppose $e \overset{\bullet}{\in} \mathcal{V}_i$. By Definition 5, there exists $\hat{e} \in \mathcal{V}_i$ such that $\hat{e}.\mathtt{b} = e.\mathtt{b}$ and $e \preceq_{\mathtt{b}} \hat{e}$. Since $\mathcal{V}_i$ contains only $e^\star$ for builder $\mathtt{b}$, we have $\hat{e} = e^\star$. Hence $e \preceq_{\mathtt{b}} e^\star$. By the definition of local eligibility, eligible bids from the same builder have contiguous sequence numbers up to the locally active bid. Therefore, every bid preceding $e^\star$ is also in $\mathcal{E}_i$, and so $e \in \mathcal{E}_i$.

We now show the reverse direction. Suppose $e \in \mathcal{E}_i[\mathtt{b}]$. Since $e^\star$ is the maximal-sequence bid in $\mathcal{E}_i[\mathtt{b}]$, we have $e \preceq_{\mathtt{b}} e^\star$. Since $e^\star \in \mathcal{V}_i$, by Definition 5, $e \overset{\bullet}{\in} \mathcal{V}_i$.

The argument holds independently for every builder, so for every bid e:

$$e \overset{\bullet}{\in} \mathcal{V}_i \quad \Longleftrightarrow \quad e \in \mathcal{E}_i.$$

□

Definition 6. *Threshold-supported eligible set: For a committed FBB value* Val*, let* $\mathcal{C}(\mathrm{Val})$ *be the collection of summaries extracted from* Val*. We define*

$$\mathsf{Supp}_{2f+1}(\mathrm{Val}) = \left\{ e \;\middle|\; \left| \{\mathcal{V}_j \in \mathcal{C}(\mathrm{Val}) : e \overset{\bullet}{\in} \mathcal{V}_j\} \right| \geq 2f+1 \right\}.$$

as the threshold-supported eligible set.

Lemma 6. *Interpretation soundness: For every committed FBB evidence value* Val*,*

$$\mathsf{AppInterp}(\mathrm{Val}) = \mathsf{AppCompress}(S(\mathrm{Val})).$$

Equivalently, $\mathsf{AppInterp}(\mathrm{Val})$ *is the canonical active-bid summary of all bids represented by at least* $2f+1$ *summaries in* $\mathcal{C}(\mathrm{Val})$*.*

Proof. Let $\mathcal{A} = \mathsf{AppInterp}(\mathrm{Val})$ be the output of Algorithm 2. We prove that $\mathcal{A} = \mathsf{AppCompress}(S(\mathrm{Val}))$ builder by builder.

Fix a builder $\mathtt{b}$. Let $T_{\mathtt{b}}$ be the multiset of active bids from $\mathtt{b}$ appearing in the summaries of $\mathcal{C}(\mathrm{Val})$. Order $T_{\mathtt{b}}$ in non-increasing sequence number:

$$e_1, e_2, \ldots \quad \text{where} \quad e_1.\mathtt{seq} \geq e_2.\mathtt{seq} \geq \cdots.$$

If $|T_{\mathtt{b}}| < 2f+1$, then no bid from $\mathtt{b}$ can be represented in at least $2f+1$ summaries. Hence $S(\mathrm{Val})[\mathtt{b}] = \emptyset$, and both $\mathsf{AppCompress}(S(\mathrm{Val}))$ and Algorithm 2 output no active bid for $\mathtt{b}$.

Otherwise, let $e^\star = e_{2f+1}$ be the $(2f+1)$-th bid in this order. By Definition 5, a bid e from builder $\mathtt{b}$ is represented by a summary containing active bid $\hat{e}$ exactly when $e \preceq_{\mathtt{b}} \hat{e}$. Therefore, e is represented by at least $2f+1$ summaries if and only if at least $2f+1$ bids in $T_{\mathtt{b}}$ have sequence number at least $e.\mathtt{seq}$.

It follows that the maximum-sequence bid in $S(\mathrm{Val})[\mathtt{b}]$ is exactly $e^\star$. Indeed, $e^\star$ is represented by the first $2f+1$ summaries in the order, so $e^\star \in S(\mathrm{Val})$. Any bid e with $e^\star \prec_{\mathtt{b}} e$ has sequence number larger than $e^\star$, and is therefore represented by fewer than $2f+1$ summaries; hence $e \notin S(\mathrm{Val})$.

Thus, for builder $\mathtt{b}$, $\mathsf{AppCompress}(S(\mathrm{Val}))$ keeps exactly $e^\star$. This is also exactly the bid selected by Algorithm 2. Since the argument holds for every builder independently, we conclude

$$\mathsf{AppInterp}(\mathrm{Val}) = \mathsf{AppCompress}(S(\mathrm{Val})).$$

□

Lemma 7. Π_{DPaaS} *satisfies Censorship-Resistance: for any builder* $\mathtt{b}$*, if its bid* e *satisfies, for all honest* PE_i*,* $e \in \mathcal{E}_i$*,*

after executing $\Pi_{\textit{Crab-BB}}$ *in* Π_{DPaaS} *with compressed input and output interpretation defined in Algorithm 2,* e *should satisfy* $e \overset{\bullet}{\in} \mathcal{A}$.

Proof. The soundness Lemma 5 and Lemma 6 shows Π_{DPaaS} is in fact an instantiation of $\Pi_{\mathsf{Crab\text{-}BB}}$. By Theorem 3 showing that $\Pi_{\mathsf{Crab\text{-}BB}}$ is censorship-resistant, we know the Π_{DPaaS} is also censorship-resistant. □

Lemma 8. Π_{DPaaS} *satisfies Backed-Availability: for any builder* $\mathtt{b}$*, if its bid* $e \overset{\bullet}{\in} \mathcal{A}$*, then there must exist* $f+1$ *honest* PE_i *backing* $e \in \mathcal{E}_i$.

Proof. The soundness shows Π_{DPaaS} is in fact an instantiation of $\Pi_{\mathsf{Crab\text{-}BB}}$. By Theorem 3 showing that $\Pi_{\mathsf{Crab\text{-}BB}}$ satisfies backed-availability, we know the Π_{DPaaS} satisfies backed-availability. □

Theorem 5. *If* GST $< T_e - \Delta_{b2p} - \Delta_{pr}$*, Cancellation Effectiveness (Goal 5, Def. 3.6) always holds in the protocol* Π_{DPaaS}.

Proof. Fix any builder $\mathtt{b}$. Let e be the most recent bid from $\mathtt{b}$ observed after GST and before $T_e - \Delta_{b2p} - \Delta_{pr}$. By the post-GST latency bound Δ_{b2p} and processing bound Δ_{pr}, bid e is delivered to and processed by every honest PE before T_e. Hence $e \in \mathcal{E}_i$ for every honest PE_i. By Censorship-Resistance of Π_{DPaaS} (Lemma 7), we have $e \overset{\bullet}{\in} \mathcal{A}$. By the DPaaS representation relation, this implies that $\mathcal{A}$ explicitly contains some bid $\hat{e}$ from the same builder with $e \preceq_{\mathtt{b}} \hat{e}$. Therefore, for every earlier bid $e' \in \mathcal{E}[\mathtt{b}]$ such that $e' \prec_{\mathtt{b}} e$, we have $e' \prec_{\mathtt{b}} \hat{e}$.

Winner finalization selects only explicit active bids in $\mathcal{A}$. Therefore e' cannot be selected as the winner. Since this holds for every earlier bid e' and every builder $\mathtt{b}$, Bid Cancellation Effectiveness holds. □

Remark on Theorem 5. Consequently, the Bid Cancellation Effectiveness guarantees a builder that after GST, those builders who submit their bids to all PEs before $t - \Delta_{b2p} - \Delta_{pr}$, can always ensure that PEs will only consider their most recent bid and not the stale one that should be canceled. Such a guarantee is missing with existing relays since cancellation is best-effort and thus a relay could effectively censor bids. However, if the builder still submits new bids after $t - \Delta_{b2p} - \Delta_{pr}$ or before GST, the cancelation may only happen on a best-effort basis.

Theorem 6. *After* GST*, protocol* Π_{DPaaS} *guarantees Bid Selection (Goal 1, Def. 3.1).*

Proof. Let $\mathcal{A}$ be the globally active bid summary output by Π_{DPaaS}, and let w be the bid selected by the winner-finalization rule.

First, we prove selection of the highest globally eligible bid. Let e be a bid such that, for every honest replica PE_i, we have $e \in \mathcal{E}_i$. By Censorship-Resistance of Π_{DPaaS} (Lemma 7), after executing $\Pi_{\mathsf{Crab\text{-}BB}}$ with DPaaS compression and interpretation functions, bid e is represented in the final active summary:

$$e \overset{\bullet}{\in} \mathcal{A}.$$

By the DPaaS representation relation, this means that $\mathcal{A}$ contains an explicit active bid $\hat{e}$ from the same builder such that

$$e \preceq_{\mathtt{b}} \hat{e}.$$

Thus, the builder of e has an active representative in $\mathcal{A}$, and any earlier bid from the same builder is considered canceled.

Now suppose that e has the highest bid value among all globally active bids at the end of the auction. By interpretation soundness, $\mathcal{A}$ is exactly the canonical active summary produced from the protocol-induced globally eligible set. Therefore, the winner-finalization rule selects the explicit bid with the largest bid value in $\mathcal{A}$:

$$w = \arg\max_{a \in \mathcal{A}} a.\mathtt{bv}.$$

Since e is represented by the active bid for its builder and has the highest value among globally active bids, the selected winner is exactly that highest globally active bid.

Second, we prove backed availability of the winner. Since w is selected from $\mathcal{A}$, we have

$$w \overset{\bullet}{\in} \mathcal{A}.$$

By Backed-Availability of Π_{DPaaS} (Lemma 8), there exists at least $f+1$ honest replicas PE_i such that

$$w \in \mathcal{E}_i.$$

Therefore, the winning bid is backed by at least $f+1$ honest PEs. Since honest PEs retain the corresponding bid data needed for reconstruction, the winner can be reconstructed.

Hence, after GST, Π_{DPaaS} both selects the highest globally active bid and ensures that any selected winner is backed by enough honest replicas to be reconstructed. □

Remark on Theorem 6. The guarantee ensures two aspects. First, it grants the builder clear guarantee that if it reaches all honest PEs and the builder did not cancel it, it must be considered as a candidate for winner. Second, it grants the proposer that the winner's block data must be backed by at least $f+1$ honest PEs. This theorem directly implies the correctness of the DPaaS protocol.

Theorem 7. *Block Availability (Goal 3, Def. 3.3) always holds in* Π_{DPaaS}.

Proof. By Theorem 1, $\Pi_{\mathsf{Crab\text{-}BB}}$ ensures all PEs terminate eventually. By Theorem 6, if any bid w is selected as the winner, w should be within the globally active bid set and must be backed by $f+1$ honest PEs. As the threshold for both secret share and BLS signature is $f+1$ in Π_{rec}, the $f+1$ honest PEs will output the Signed Beacon Block. □

Theorem 8. *If* Π_{vbb} *can achieve 2-rounds good-case latency,* Π_{DPaaS} *can achieve the good-case latency of* 5 *rounds to*

determine the winner, and can achieve the optimistic (all PE*s are honest) good-case latency of* 4 *rounds. (Goal 6)*

Proof. Since DPaaS uses $\Pi_{\mathsf{Crab\text{-}BB}}$ as a primitive $\mathsf{T}_{gc}(\Pi_{\mathsf{DPaaS}}) = \mathsf{T}_{gc}(\Pi_{\mathsf{Crab\text{-}BB}})$. We instantiated Π_{vbb} by $(5f-1)$-psync-VBB which can achieve 2-rounds good-case latency by Lemma 4, so $\mathsf{T}_{gc}(\Pi_{vbb}) = 2$. Considering the two rounds of exchange $\tau_{ex} = 2$ added in $\Pi_{\mathsf{Crab\text{-}BB}}$, at most 4 rounds are needed to commit the final winner. In total, including one round for bid submission as in previous work [79], the good-case latency is $\mathsf{T}_{gc}(\Pi_{\mathsf{DPaaS}}) = 5$.

Under optimistic good-case, if a valid message `ex-fast-2` exists in the network, every honest PE can get prepared with one signed `ex-fast-2` message. This only takes 1 round ($\tau_{ex} = 1$) for all honest PEs to complete then 2 rounds after entering Π_{vbb}. Therefore, the optimistic good-case latency is $\mathsf{T}_{gc}(\Pi_{\mathsf{DPaaS}}) = 4$. □

Remark. This result matches the lower bound shown in [79] when the Byzantine fraction is larger than $\frac{1}{5}$ (when $n = 4$), but has one extra round for good-case latency when n is larger. We leave more discussion to Appendix D.

Theorem 9. *Timely Block Availability (Goal 3, Def. 3.4) holds after* GST *before* T_p.

It's easy to show Theorem 9 holds given Theorem 7 after GST.

Remark. Besides the theorem, it is worth exploring how to set such T_p, because setting T_p too conservative ($T_p = 4$s) may incur a missed slot; setting T_p too aggressive may end the auction too early thus missing MEV opportunity.

By Theorem 8, we know that the good-case post-auction latency can be $5 - 1 = 4$ rounds plus reconstruction latency and optimistic good-case post-auction latency can be $4-1 = 3$ rounds plus reconstruction latency by excluding the bidding phase.

In both cases, Δ for the first round of exchange is necessary. In optimistic good-case, all honest PEs can directly enter Π_{vbb} without waiting for one more round. In other good-case, δ is added to collect the $f+1$ $\mathcal{V}$. Due to the responsiveness of Π_{vbb}, Π_{vbb} takes only 2δ to finish in the good case. At reconstruction, as reconstruction and interpolation can be parallelized to one round, we denote it as δ_{rec}. To sum them up, the optimistic good-case (best case) post-auction will cost $\Delta + 2\delta + \delta_{rec}$ and normal good-case will cost $\Delta + 3\delta + \delta_{rec}$.

But if the first leader entering Π_{vbb} is a faulty leader, Π_{vbb} cannot hit good-case if the faulty leader is silent or proposes some invalid proposal. Thus, Π_{vbb} may not terminate until the first honest leader appears. If the leaders are rotated round-robin, and the Π_{vbb}'s view-length is 4Δ (as in $(5f-1)$-psync-VBB), this may take $4f\Delta + 2\delta$ to terminate. Therefore, the worst-case post-auction will cost $\Delta + 4f\Delta + 3\delta + \delta_{rec}$. When $n = 4, f = 1$, this actually gives us the lowest worst-case latency $5\Delta + 3\delta + \delta_{rec}$.

APPENDIX D
MORE RESULTS ON $\Pi_{\mathsf{Crab\text{-}BB}}$

In this section, we touch on a more interesting result for $\Pi_{\mathsf{Crab\text{-}BB}}$. We can show several ways to reduce one round to achieve consensus and argue the protocol shown in Fig. 3 meets the best trade-off between latency and fault tolerance.

From the foundational work [78], we know that: "any protocol for censorship-resistance in partial synchrony secure against more than $\frac{1}{5}$-fraction of Byzantine faults requires 5 rounds". According to Theorem 8, $\Pi_{\mathsf{Crab\text{-}BB}}$ matches this lower bound when $n = 4$ under $n = 5f - 1$.

However, we can match this lower bound given larger n (*i.e.*, $f \geq 2$ when $n = 5f - 1$). The solution is to remove the **Exchange-2** step in the Fig. 3 and set the external validity of Π_{vbb} to just one WQ. It's trivial to show the Agreement, Termination, and Non-triviality. We can show this is sufficient for censorship-resistance and backed availability defined in Definition 1.

For censorship-resistance, the size of WQ is at least $n-f = 4f - 1$. For any input u made to all honest replicas, u is guaranteed to be implicitly in at least $4f - 1 - f = 3f - 1$ compressed input set. As $f \geq 2$, $3f - 1 \geq 2f + 1$, which implies such input u must be included to output interpretation subsequently. For availability-backed, the argument directly follows Theorem 3 as we didn't change the output interpretation in this optimization.

On the other hand, intuitively, we can show why fewer rounds is impossible for $f = 1$. First, the builder submission should be treated as one round and under good-case, the Π_{vbb} still need to take 2 rounds to finish. The key is that the two exchanging round can't be reduced to one because a simple WQ is not sufficient for censorship-resistance due to the number of honest replicas' input in WQ is less than the threshold of inclusion in output interpretation because $n - f - f < 2f + 1$ when $f = 1$. When the leader is faulty, it is able to insert stale bids (with smaller sequence numbers) to exclude honest replicas' input, and thus censoring certain bids.

Moreover, adding one more replica to $n = 4$ to $n = 5$ then assuming at most 1 can be faulty can also reduce one round to match the 4 round good-case latency. The key idea is still to make the WQ contain sufficient shares $(2f + 1)$ from honest replicas.

From a practical standpoint, the cost of renting TEEs creates a strong incentive to use the smallest committee size that still provides meaningful fault tolerance. Moreover, the limited diversity of availability zones and TEE implementations in practice makes it impractical to instantiate too many PEs without weakening the underlying trust assumptions. Our choice of $n = 4$ and $f = 1$ reflects this practical design point to get the best of the two worlds: it is the smallest deployment that tolerates the highest fraction of faulty TEE while achieving optimal latency. Though adding more parties can reduce one round, but this can greatly increase the operational cost and may break the isolated failure assumed by the protocol.

Leader ID	Crashed ID: None	0	1	2	3
0	41.13±3.13	155.91±14.42	67.21±9.17	86.29±3.22	74.45±9.98
1	40.81±5.26	86.12±3.24	148.38±12.51	86.72±3.13	88.31±5.30
2	42.88±4.56	87.36±4.49	69.13±8.22	165.13±18.03	71.31±4.63
3	40.69±5.88	85.47±4.09	65.28±5.11	87.13±2.66	146.31±15.78

TABLE IV: Average latency (ms) ± std.dev for calling Crab-BB under different combination of leader and crashed party choice when $\Delta = 23$ ms. **None** means no faulty PEs. All in millisecond (ms). Case 1: Every PE is honest; Case 2: First view follower crashed; Case 3: First view leader crashed, time out needed.

Appendix E
Post-Auction Performance: Consensus Details

As shown in Table IV, in Case 1 refers to the "fast path", the first leader can form SQ and will then enjoy good-case latency, bounded by 3Δ rounds in total. In Case 2, one of the followers in the first view crashed at the start, which can drive the protocol away from fast path. Therefore, first leader will wait for at most two Δ at the start then enter the VBB protocol with a normal-path proposal, whose upper bound is 4Δ. In Case 3, the first leader crashed therefore a view change after a timeout of 4Δ and the second view can make the other honest PEs commit in good-case latency, 8Δ in total.

Appendix F
Macrobenchmark: Bid Count

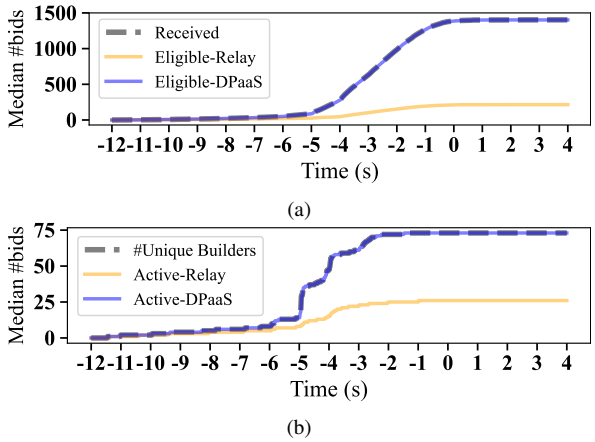

Fig. 10: Median number of eligible (a) and active (b) bids throughout the auction timeline for a slot, comparing DPaaS and Relay against upper bounds (received bids and unique builders).

During the auction, both the count of eligible bids (#EB(t)) and active bids (#AB(t)) matters. For #EB(t), the closer it matches the number of received bids at time t, and for #AB(t), the closer it matches the number of active builders at time t, the more efficient the bid processing is. We measured #EB(t) and #AB(t) for both the relay (using historical bid traces) and DPaaS (by simulating the auction with the same traces). As shown in Fig. 10, the Ultra Sound Relay in December 2023 exhibited significantly lower #AB(t) and #EB(t) throughout the slot compared to DPaaS, which removes bid validation from the critical path. This discrepancy arises because Ultra Sound Relay had adopted the optimistic-v1 [76] mode since September 2023. In this mode, the relay fully simulates only bids from builders outside the allowlist (those without stake on the relay), while merely performing lightweight checks for allowlisted builders and marking their bids as eligible. Consequently, many bids remain queued for delayed simulation, and over 50 builders' bids are effectively ignored at certain times t.

In contrast, DPaaS substantially improves both #AB(t) and #EB(t), even in a decentralized setting where eligibility and activity require a threshold of at least $2f + 1$ parties. This demonstrates that by integrating BT into DPaaS, the scalability challenges faced by relays under heavy bid loads can be addressed—without resorting to optimistic modes or requiring builder to stake on the relay.

Listing 1 Key structures and interfaces for SC_R

```
contract DPaaSRegistry {
    struct PEInfo {
        string endpoint;
        bytes  blsPubkey;
        bytes  asymPubkey;
        string provider;
        string zone;
        bytes  blsSig;
    }
    struct Deployment {
        address  validator;  // ETH withdrawal
        ↪ address of the validator
        PEInfo[] pes;
        bytes32  manifestHash;
        bytes    aggregatedPubkey;
        bytes    aggregatedSigOverManifest;
    }
    mapping(bytes32 => Deployment) public
    ↪ deployments;
    function register(
        bytes     calldata validatorBLSPubkey,
        address            validatorAddress,
        PEInfo[]  calldata pes,
        bytes32            manifestHash,
        bytes     calldata aggregatedPubkey,
        bytes     calldata
        ↪ aggregatedSigOverManifest,
        bytes     calldata sig,
        bytes     beacon_proof
    )
    function deregister(
        bytes     calldata validatorBLSPubkey,
        bytes     calldata sig,
    )
    function getPEManifest(bytes calldata
    ↪ validatorBLSPubkey)
        external view
        returns (bool found, PEInfo[] memory pes)
}
```

Appendix G
Smart Contracts

Listing 2 Key structures and interfaces for SC_E

```
contract OperatorClaim {
    // EIP-4788 Beacon Roots Contract
    address constant BEACON_ROOTS =
    ↪ 0x000F3df6D732807Ef1319fB7B8bB8522d0Beac02;
    struct Escrow {
        address operator;
        bytes32 validatorPubkeyHash;
        uint256 reward;
        bool    claimed;
    }
    mapping(bytes32 => Escrow) public escrows;
    function claim(
        bytes32   escrowId,
        uint64    timestamp,
        bytes32[] calldata pubkeyProof
    ) external {
        /// EIP-4788: fetch the beacon block root
        ↪ committed at `ts`.
        bytes32 beaconRoot = _beaconRoot(timestamp);
        require(
        /// Using SSZ.
        _verifyValidatorPubkey(beaconRoot,
        ↪ escrow.validatorPubkeyHash,
        ↪ pubkeyProof),
            "bad pubkey proof"
        );
        ...
        escrow.claimed = true;
        (bool ok, ) = msg.sender.call{value:
        ↪ escrow.reward}("");
        ...
    }
}
```

In this section, we provide a high-level overview for the smart contracts that we develop and leverage in DPaaS, including the data structures and function interfaces.

Registry Contract. Listing 1 shows basic structure of DPaaS deployment's metadata and interface for SC_R.

We require both validator deposits and DPaaS registration to be performed by the user controlling the withdrawal key $(sk_{\mathrm{W}}, pk_{\mathrm{W}})$. Accordingly, in the registration contract SC_R. The `register` verifies an ECDSA signature included in the call data. The contract will check (1) if the ECDSA signature is from a valid validator; (2) if the validator's withdrawal address is tied to the validator public key in the beacon state. The contract also provides the `deregister` call for the user. Given the (2) is checked in the `register`, the `deregister` only checks if the ECDSA signature is from a valid validator and if the withdrawal credential matches the address of validator in the deployment.

To change the registered deployment metadata, the user must first deregister and then submit a new registration. This prevents unauthorized or silent modification of the public registry state.

For builder side verification, the builder can get the manifest file for any DPaaS validator from SC_R. Note that such read operation doesn't cost gas as they can be done in the builder's local synced node because of the `view`. If a validator is not enrolled through DPaaS, then the builder will get an empty record from the storage. We consider a fake registration (*i.e.*, a non-DPaaS validator register itself as a DPaaS validator) as a behavior that only hurts the validator itself, as this require gas cost but will not pass the builder verification.

Escrow Contract. Listing 2 shows a simplified version of SC_E and its claim procedure. After a DPaaS proposer successfully proposes a block through the operator's deployment, the operator may claim the corresponding reward. The operator address is fixed at contract deployment, so only the registered operator can submit claims. To claim, the operator provides the block timestamp and a SSZ proof [87] tied to the proposer's public key. The contract verifies the corresponding beacon root using EIP-4788 [88] and enforces that each proposed block can be claimed only once.